

Beyond Prompting: An LLM-Driven Workflow for Code Clone Refactoring

Xing Qian^a, Amy Arias Ramirez^a, Eman Abdullah AlOmar^{a,*}

^a*Stevens Institute of Technology, Hoboken, NJ, USA*

Abstract

Context: Code clones are common in software systems and often lead to maintenance and bug-related issues. Extract Method is a standard refactoring technique to remove duplication, but applying it correctly requires preserving behavior, compiling successfully, and satisfying structural constraints. Recent studies show that single-prompt Large Language Models (LLMs) can generate syntactically correct refactorings, yet they frequently produce incomplete or structurally invalid transformations, especially for Type-2 and Type-3 clones.

Objectives: This paper investigates whether a structured LLM driven workflow can improve the reliability of clone refactoring beyond single-prompt generation. In particular, we study how staged validation, collaborative agent interactions, retrieval augmentation, and structural usefulness checking contribute to the correctness and behavioral stability of Extract Method refactorings on real-world code clones.

Methods: In this paper, we present a LLM driven workflow for clone refactoring. The workflow decomposes the task into structured stages, including clone detection, refactoring generation, structural usefulness checking, compilation validation, and test-based behavioral verification. Across the detection, refactoring, and usefulness-checking stages, we employ multiple LLM-based agents to evaluate candidate outputs before proceeding to the next stage. We also incorporate retrieval-augmented generation (RAG) to guide the agents with curated Extract Method examples, while a PSI-based usefulness checker further enforces structural correctness beyond compilation and testing.

Results: We evaluate the workflow on real-world clone instances collected from the ArgoUML project using Deckard. Results show that the workflow achieves 93.1% detection, 90.0% usefulness, 80.0% compilation success, and 69.2% test success overall on real-world clones. The results also show that clone size has a noticeable impact on refactoring stability. Medium-sized clones generally achieve the best balance between extraction context and structural manageability, while large clones remain more challenging due to increased de-

*Corresponding author

Email addresses: xqian7@stevens.edu (Xing Qian), aariasra@stevens.edu (Amy Arias Ramirez), ealomar@stevens.edu (Eman Abdullah AlOmar)

pendency interactions and structural complexity.

Our ablation study further shows that retrieval-augmented generation (RAG) plays an important role in improving behavioral stability during refactoring. Removing RAG causes the largest decrease in overall test success, reducing the success rate from 70.0% to 30.8%. In contrast, removing the usefulness checker, feedback loop, or collaborative panelist architecture leads to smaller but still noticeable reductions in structural correctness and overall workflow reliability.

Conclusion: Our findings suggest that reliable LLM-based refactoring requires structured decomposition, explicit structural validation, and iterative feedback, rather than relying on one-shot code generation.

1. Introduction

Nowadays there are a lot of code clones in software development as developers can save time by just copying and pasting the reused logic. Although this action can help speed up the development, it could also introduce bugs and issues for maintenance. For example, if the same code fragments exist in multiple places in the project, when developers make changes to them, those changes should be applied to every copy. Missing one copy could cause inconsistency problems and make the project harder to maintain in the future [34, 21].

To solve the issues that cloned codes may bring, it is important to refactor those codes. Code refactoring refers to the process of improving structures without changing internal behavior [17]. There are many different code refactoring methods, among them, the Extract Method refactoring is one of the most used technique to address code clone issues [17, 25, 7]. Extract Method refactoring moves the cloned codes into a single method, and then replaces the original clones with shared method calls. However, when applying the Extract Method refactoring, developers need to make sure it preserves the original functionalities and compiles without any issues, which makes the refactoring not easy.

As Large Language Models (LLMs) emerged fast during these years, researchers have begun to study whether LLMs can assist with code clone detection [41, 16] and refactoring [11, 22]. Recent models from OpenAI and other providers can generate code and understand programming patterns. A recent empirical study [30] evaluated GPT-4o’s ability to classify and refactor code clones in a real-world project. Their study found that LLMs performed well on simple clones. However, when the clones became more complex, the performance dropped significantly. They also found several LLMs hallucinated patterns regarding Extract Method refactorings. The refactorings generated by LLMs do not always follow the definition of Extract Method, even though the models are explicitly instructed, so the extraction patterns are not always useful. In their work, they found patterns such as incomplete refactoring, directly deleted clones, fragmented logic into multiple small methods, or extracted more code than necessary. For example, Listing 1 shows one example from the empirical study [30]. In line 30, the refactoring proposed by LLM directly replaced the cloned method `init_Cloned()` with a method call to the original method `init()`.

These findings suggest that a single LLM [that uses one-shot generation](#) which cannot reliably handle the complete detection and correction of code clone. Refactoring requires multiple steps: detecting clones, planning the refactoring, generating code, checking compilation, and running tests. Multi-agent systems [27, 40, 10, 35] offer a new direction for clone detection and refactoring. Instead of using a single LLM for the entire task, we can assign different roles to specialized agents. Each agent focuses on a specific task and uses tools to interact with the development environment. This design has been successful in other software engineering tasks. For example, RefAgent [27] uses four specialized agents for refactoring and achieves a 90% unit test pass rate through iterative feedback loops.

Listing 1: Incorrect Extract Method refactoring example

```

1 public void init() {
2     createIconkit();
3     getVersionControlStrategy().assertCompatibleVersion();
4     setUndoManager(new UndoManager());
5
6     getContentPane().setLayout(new BorderLayout());
7
8     fView = createDrawingView();
9
10    JPanel attributes = createAttributesPanel();
11    createAttributeChoices(attributes);
12    getContentPane().add("North", attributes);
13
14    JPanel toolPanel = createToolPalette();
15    createTools(toolPanel);
16    getContentPane().add("West", toolPanel);
17
18    getContentPane().add("Center", (Component)view());
19    JPanel buttonPalette = createButtonPanel();
20    createButtons(buttonPalette);
21    getContentPane().add("South", buttonPalette);
22
23    initDrawing();
24    // JFC should have its own internal double buffering...
25    //setBufferedDisplayUpdate();
26    setupAttributes();
27 }
28
29 public void init_Cloned() {
30     init();
31 }

```

Inspired by these approaches, we propose a LLM driven framework specifically designed for code clone detection and refactoring. Our framework is triggered when a developer pastes code, inspired by the AntiCopyPaster tool [4, 5,

6, 3].

Our framework addresses the limitations observed in previous work through several designs. Instead of relying on a single LLM to do both detection and refactoring, we adopt a LLM driven workflow. In our setting, we design several agents where each one is responsible for one specific task.

First, we decompose the clone detection and refactoring process into specialized agents. The Detection Agent identifies clone candidates using LLM reasoning and a PSI-based fallback. The Refactoring Agent generates Extract Method refactorings guided by retrieval-augmented examples. The Usefulness Checker evaluates whether the generated refactoring truly removes duplication through a meaningful Extract Method transformation. The Compilation validator validates syntax correctness, and the Testing validator checks behavior preservation using auto-generated tests from EvoSuite [18]. In addition, we use LLM-based panelists and a curator agent across the detection, refactoring, and usefulness-checking stages. The panelists provide multiple independent judgments or candidate outputs, while the curator aggregates these results and selects the most reliable decision before the workflow proceeds to the next stage. Unlike EM-Assist [29, 28], which mainly concentrates on validating and filtering LLM-generated refactoring candidates, our approach models clone refactoring as an end-to-end workflow beginning from clone detection. The workflow combines clone detection, panelist consensus, PSI-based fallback detection, usefulness checking, compilation validation, testing validation, and iterative feedback-driven repair into a unified multi-stage process for Extract Method refactoring.

Second, we introduce a usefulness checker that explicitly filters out the problematic refactoring patterns identified in a prior empirical study [30]. Besides LLM-based panelists and curator, this checker also uses Program Structure Interface (PSI)-based structural analysis and token similarity to detect different types of unuseful outcomes: incomplete refactoring, excessive refactoring, post-extraction clone deletion, direct clone removal, call-based clone substitution, clone removal by delegation, and fragmentation of logic. Only refactorings that successfully compile, pass all tests, satisfy the usefulness check, and are approved by human developers are applied to the code.

Third, we employ retrieval-augmented generation (RAG) to provide the Refactoring Agent with concrete examples from correct Extract Method refactorings. This approach grounds the LLM’s generation process in demonstrated patterns rather than relying solely on its training data.

Importantly, the agents in our workflow do not do each task independently. Instead, we include a feedback loop. If the later component such as Usefulness Checker, Compilation validator, and Testing validator fail, they will send the error message back to Refactoring Agent, which allows the workflow to refine the results iteratively.

We evaluate our approach on 120 clone cases from the Java project ArgoUML. Among these 120 cases, we cover Type-1, Type-2, and Type-3 clones. Here are the definitions for each clone type [33].

- Type-1: exact copies, except for whitespace, comments, and layouts.

- Type-2: only differ in identifiers, literals, or data types.
- Type-3: mostly similar but include added, removed, or modified statements.

We first report the effectiveness of the proposed LLM driven workflow for code clone detection and refactoring using real-world clones (RQ1). Then, we conduct an ablation study to assess the contribution of individual components, including retrieval-augmented generation, usefulness checking, and feedback-based iteration (RQ2). Since Deckard may produce false positives and does not explicitly report clone types, we manually inspect and classify the detected clone instances into Type-1, Type-2, and Type-3 clones before evaluation. We further analyze the performance of the workflow across different clone sizes, including small, medium, and large clones.

This paper makes the following contributions:

- We designed a LLM-driven framework for code clone detection and Extract Method refactoring that combines LLM-based panelists, curator, and a PSI-based fallback mechanism to improve detection grounding and refactoring reliability.
- We introduce a usefulness checker based on PSI analysis together with LLM-based usefulness panelists and a curator agent to detect, validate, and filter out unuseful refactoring patterns observed in prior work [30].
- We implement retrieval-augmented generation that provides clone detection and refactoring examples to guide the LLM during both detection and refactoring stages.
- We conduct an empirical evaluation on both manually constructed and real-world clone instances, including Type-1, Type-2, and Type-3 clones with different clone sizes collected from [ArgoUML](#).

The rest of this paper is organized as follows. Section 2 describes our problem statement. Section 3 reviews related work. Section 4 presents our LLM-driven framework. Section 5 reports experimental results. Section 6 discusses our findings. Section 7 addresses threats to validity, and Section 8 concludes. Our project with clones, code and results can be found in the replication package [1].

2. Problem Statement

Large Language Models (LLMs) have recently been applied to code clone detection and automated refactoring [20, 30, 29, 28]. While these models can generate refactorings that often compile and pass tests, their behavior is still unreliable. In practice, a single LLM invocation frequently produces incorrect clone detection results or refactorings that preserve behavior but fail to achieve a meaningful Extract Method transformation.

One key difficulty is that clone refactoring involves multiple types of reasoning. The system must first correctly detect duplicated code. It must then apply a structurally sound refactoring. Finally, it must ensure that the refactoring preserves program behavior. A single LLM prompt is rarely sufficient to handle all of these steps in a reliable way.

To address this challenge, we design a LLM driven workflow that separates clone detection, refactoring, and usefulness into distinct agents, along with compilation and testing validators. This workflow allows intermediate validation and feedback, rather than relying on one-shot generation. Based on this design, we study the following research questions.

RQ1: How effective is the proposed workflow for clone detection and Extract Method refactoring in terms of correctness and usefulness when applied to real-world code clones? Our first research question evaluates the effectiveness of the proposed LLM driven workflow on real-world code clones. We study whether the workflow can correctly identify code clones and generate refactorings that are both valid and useful. Correctness refers to whether the refactored code successfully compiles and passes all tests. Usefulness refers to whether the refactoring represents a meaningful Extract Method transformation, rather than incomplete refactoring, excessive extraction, or other degenerate cases.

To answer this question, we collect clone instances from the ArgoUML project [32]. We select candidate subject systems from previous systematic literature reviews on clone detection and refactoring [24]. After applying Deckard to these projects, we found that ArgoUML contains the largest number of clone groups, making it the most suitable subject system for our real-world evaluation.

To further analyze the impact of clone size, we divide the selected clones into three categories based on their length: small clones (1–5 lines), medium clones (6–10 lines), and large clones (more than 10 lines). For each category, we evaluate 20 clone instances.

This research question allows us to assess whether the workflow generalizes to realistic development scenarios and how its performance varies across different types of real-world clones.

RQ2: What is the contribution of key validation and guidance components in the LLM driven framework? Our second research question focuses on understanding the role of several core components that support the LLM driven workflow. We design targeted ablation settings that disable specific validation or guidance mechanisms. In particular, we evaluate three variants: (1) removing retrieval-augmented generation (No RAG), (2) removing the usefulness checker (No Usefulness), (3) disabling the feedback-based iteration mechanism (No Feedback Loop), and (4) removing panelists and curator (No Panelists).

These components are not independent agents, but cross-cutting mechanisms that influence how refactorings are generated, filtered, and refined. By

selectively disabling them while keeping the overall pipeline intact, we can observe how each mechanism affects end-to-end refactoring success, usefulness, compilation correctness, and test pass rates.

This ablation study allows us to identify which validation and guidance mechanisms are essential for reliability and which mainly provide incremental improvements.

Scope of Clone Types. This study focuses on Type-1, Type-2, and Type-3 clones. We do not include Type-4 clones in our evaluation. Type-4 clones refer to code fragments that implement the same functionality using substantially different implementations. Detecting and refactoring such clones requires deep semantic understanding of program intent, project architecture, class dependencies, and design constraints.

Together, these research questions provide a structured evaluation of the proposed LLM driven workflow. They help clarify whether the approach improves clone refactoring reliability, which components matter most, and how feedback-driven iteration compares to one-shot LLM-based refactoring.

3. Related Work

There exist many studies focusing on code clone detection and refactoring. Before LLMs came out, early studies had adopted static analysis techniques to refactor code clones safely without changing the behaviors. For example, Tsantalis et al. [38] proposed a systematic approach to assess clone refactorability by analyzing structural similarity, statement mapping, and refactoring preconditions. Their approach provides strong guarantees on compilation and behavioral preservation. Tools such as JDeodorant build on these principles to recommend Extract Method refactorings based on program structure and code smells.

To better integrate refactoring into developers’ daily programming, recent work has explored interactive and just-in-time clone refactoring within the IDE. For example, AntiCopyPaster introduced the idea of monitoring copy and paste actions and detecting code duplication at the moment it is introduced. The original AntiCopyPaster [4] tool recommends Extract Method refactorings based on a classifier that predicts whether a pasted fragment should be extracted. AntiCopyPaster 2.0 [6] extends this approach by introducing white-box detection and customizable rules. They allow developers to tune detection behavior according to their preferences. AntiCopyPaster 3.0 [3] further enhances the system by leveraging IntelliJ PSI for structural clone detection. In this work, they implement a customized Extract Method refactoring pipeline and integrate method name recommendation models. Together, these studies show that just-in-time clone detection can effectively reduce workflow disruption and improve the timeliness of refactoring.

Due to the rise of Large Language Models, recent studies have investigated their abilities for code clone detection as a classification problem. Khajezade et al. [20] conducted an empirical study on applying prompt-based LLMs to

detect semantic (Type-4) code clones in both mono-lingual and cross-lingual settings. Their results show that under carefully designed prompts, ChatGPT can achieve performance comparable to or better than fine-tuned deep learning models, particularly in cross-language clone detection scenarios. This work highlights the strong reasoning capability of LLMs for understanding functional similarity between code fragments. However, this line of work only focuses on clone detection task. There are no further steps after the clones are detected. As a result, issues such as refactorability, behavior preservation, compilation correctness, and test validation are outside the scope of these approaches. In contrast, our work treats clone detection as the first step in a LLM driven pipeline, where detected clones are passed to the Refactoring Agent to produce Extract Method refactorings.

In more recent studies, LLMs have been applied to refactoring tasks. EM-Assist [29, 28] demonstrates that LLM can generate refactoring suggestions that are similar to developer practices. However, they also discover that certain amount of LLM-generated suggestions are invalid, hallucinated, or not useful without strong filtering. EM-Assist addresses this issue by combining LLM-generated candidates with IDE-based static analysis to eliminate invalid or trivial refactorings before execution. Empirical studies [30] comparing LLMs-based refactoring with traditional tools further show that LLMs perform well in simple clone scenarios, but their performance drops significantly in structurally complex clones. In complex cases, LLMs often produce incomplete refactorings or other patterns that do not meet the requirement of Extract Method refactoring. There are also a large amount of refactorings that do not compile or preserve behavior.

Recent work has begun to explore LLM driven architectures for automated refactoring as a way to address the limitations of single-LLM approaches. RefAgent [27] decomposes the refactoring process into specialized agents responsible for planning, execution, testing, and iterative refinement. Their results demonstrate that agent collaboration significantly improves compilation and test success rates compared to single-agent settings. MANTRA [40] improves automated refactoring by integrating contextual retrieval, coordinated multi-agent collaboration, and self-repair mechanisms. Their work also achieves higher rates of test-passing refactorings at the method level.

New research about multi-agent LLMs demonstrates that using consensus by a panel of panelists can mitigate hallucinations for software engineering tasks. CANDOR is introduced by Xu et al. [39] to be an architecture of multi-agents to perform end-to-end JUnit test generation. The test generation process includes six kinds of agents, which respectively perform the task in the following steps: initialization, test planning, test generation, inspection, requirement extraction, oracle correction, and finally curation. Specifically, they incorporate multiple reasoning-based panelists to independently assess proposed oracles of test cases, interpreters to extract concise statements from verbose reasoning performed by panelists, and finally a curator to form the consensus. The proposed approach successfully improves the oracle correctness and mitigates hallucinations in test cases. However, this research is closely related to our work, but its focus is differ-

ent as it generates unit tests while our work focuses on other tasks such as clone detection, Extract Method refactoring generation, and usefulness assessment.

Multi-agent designs have also been applied to other programming languages and refactoring scenarios. For example, a multi-agent LLM framework for Haskell refactoring [35] introduces role-based agents for structural analysis, code smell detection, strategy planning, execution, testing, and debugging of refactoring. They show improvements in complexity, code quality, and performance through staged validation and iterative feedback. Such work demonstrates that decomposing refactoring into explicit analysis, transformation, and validation stages can improve reliability compared to monolithic generation.

In a distinct setting, CoRenameAgent [10] applies a multi-agent framework to coordinated rename refactoring, where agents infer refactoring scope from developer actions and propagate changes across a codebase using trusted IDE refactoring APIs. Together, these studies provide strong evidence that agent specialization, staged validation, and iterative feedback are effective for complex refactoring tasks.

Overall, prior works [20, 41, 11, 12] have studied code refactoring from different angles. Traditional approaches rely on static analysis and predefined rules to assess refactorability and ensure correctness. Recent LLM-based techniques show that models can do code refactoring, but their outputs are often unstable. Multi-agent refactoring systems further improve robustness by breaking refactoring into multiple roles, but they usually assume that refactoring targets are already known.

In contrast, our work structures the code clone refactoring tasks. Unlike EM-Assist, which primarily focuses on validating and filtering LLM-generated refactoring candidates to eliminate long method code smell, our workflow treats clone refactoring as a complete multi-stage process starting from clone detection. In addition to refactoring generation, our approach incorporates clone detection, panelist-based consensus, PSI-grounded fallback detection, usefulness assessment, compilation validation, testing validation, and iterative feedback-driven repair within a unified workflow. We consider clone refactoring as a step-by-step process that should be carried out by multiple specialized agents. Clone detection serves as the starting point of the workflow. Each following step such as refactoring, compilation, and testing is handled explicitly with clear responsibilities and feedback. In addition, we introduce a usefulness checker that filters out unuseful refactorings introduced in previous work [30] that do not correctly follow the Extract Method refactoring. By putting these components into one workflow, our approach brings together ideas from previous studies and treats clone refactoring as a structured step-by-step process rather than a single step.

4. Study Design

4.1. Overview

This study adopts a LLM driven workflow to automatically detect and refactor code clones while preserving program behavior. As shown in Figure 1, the

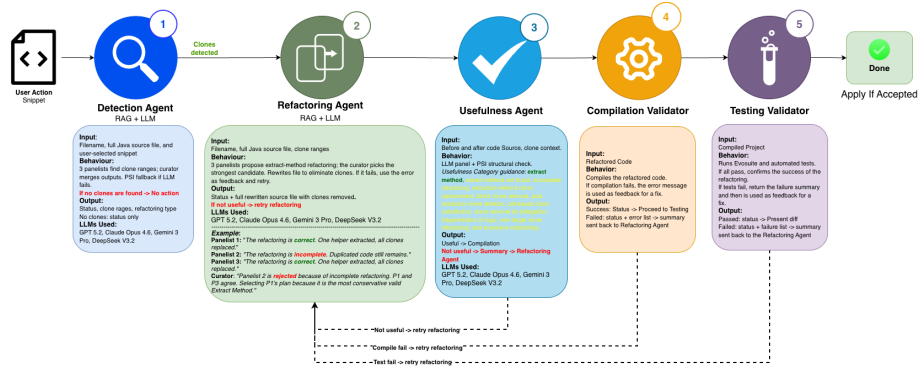


Figure 1: Workflow of LLM Driven Framework.

system first detects code clones and then generates Extract Method refactorings. Then the generated code is checked by the Usefulness Checker, Compilation validator, and Testing validator in order.

There are five major components in the pipeline: Detection Agent, Refactoring Agent, Usefulness Checker, Compilation validator, and Testing validator. Detection Agent detects the snippets using snippet-centered clone detection. It does not perform generic duplicate code detection but instead targets the pasted code snippet as the target and detects clone instances with respect to the target. Inspired by Xu et al.’s work [39], we used panelists and curator structures in detection, refactoring, and usefulness. The Detection Agent runs three LLM panelists on the same file independently, and the curating agent integrates their outputs to generate the final detection output. Since the line ranges detected via the LLM may contain some errors, the ranges detected via the LLM will be further grounded to the ground truth source code. If the LLM-powered detection process fails to detect any clones, then a conservative PSI-powered fallback detector will be used to detect clones based on text match.

Post clone detection, the Refactoring Agent produces a set of Extract Method refactoring suggestions. Like the Detection Agent, the Refactoring Agent involves three independent panelists that produce a list of refactoring proposals and a curator who chooses the most viable refactoring suggestion. Every refactoring suggestion is formulated in the form of a structured partial refactoring plan comprising the created helper method along with replacement code for every clone occurrence. The whole process is followed by implementing the refactoring suggestion on the source code.

The refactoring that was automatically produced will be assessed using the Usefulness Checker, prior to compilation or testing. This assessment ensures that the proposed refactoring is a legitimate Extract Method refactoring and not an useless refactoring like deleting a clone, keeping the duplicate code untouched, refactoring a non-clone, or creating an Extract Helper refactoring but failing to replace the cloned code. The usefulness assessment uses both LLM-based and PSI-based reasoning. In the case of whole method clones, the original

clone methods remain and call a new helper method that was created by the refactoring. For fragment clones, the duplicated fragments must have been replaced or removed by the refactoring. This step filters out unuseful refactorings identified in prior work [30] before more expensive correctness validation.

If the candidate passes the usefulness check, the Compilation validator compiles the refactored code to ensure that it builds successfully. The Testing validator then checks whether the refactoring preserves the original program behavior.

The workflow executes sequentially and will iterate when failures occur, with the maximum number of iterations controlled by user settings. When the usefulness check, compilation, or testing fails, feedback is generated and provided to the Refactoring Agent. This allows the workflow to gradually improve the usefulness and correctness of refactored code across multiple attempts.

We use three related terms to describe how refactorings are evaluated in our workflow: *valid*, *useful*, and *accepted* refactorings.

Valid refactoring. A refactoring is considered valid only if it successfully passes all automated validation steps in the workflow. Validation is performed after a refactoring proposal is generated, and it passes the Usefulness Checker.

Validation follows a simple sequence. The refactored code is first compiled using the original project classpath to ensure that it builds correctly. If compilation fails, the refactoring is rejected, and the workflow does not proceed to the testing stage.

If compilation succeeds, the workflow continues with testing. The project’s test suite is then run on the refactored code. The original project files are not modified during this step. The tests are used to confirm that the refactoring does not change the program’s behavior.

Only refactorings that pass both Compilation validator and Testing validator are considered valid. Refactorings that fail at any validation step are treated as invalid and will not be applied.

Useful refactoring. In this work, a refactoring is considered useful only if it reflects a meaningful and correct Extract Method refactoring.

In Pomian et al.’s works [29, 28], they introduced a mechanism to assess the usefulness of refactorings. We follow this idea and consider usefulness as a structural property of the refactoring. An empirical study by Qian et al. [30] observed that LLM-generated refactorings often compile and pass tests successfully but still fail to produce a proper Extract Method structure. Typical problems include incomplete refactoring, directly deleting clone code or splitting the logic into many small helper methods. Although these changes may remove duplication, they do not match the definition of a proper Extract Method refactoring. Based on these observations, we define a useful refactoring using a clear set of practical rules.

A useful Extract Method refactoring should introduce a single new helper method based on the definition, replace all clone instances with calls to this method, and remove the duplicated logic from the original locations.

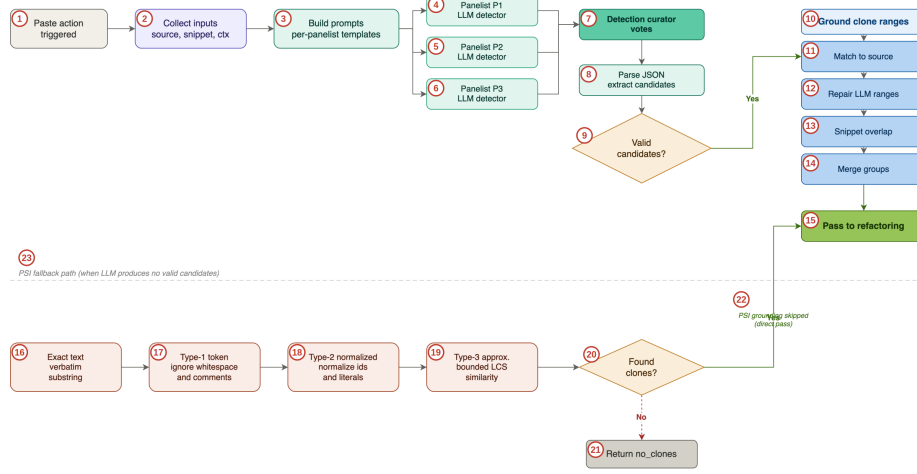


Figure 2: Detection Pipeline.

Usefulness check is carried out right after the generation of the refactoring candidate by the Refactoring Agent before compiling and testing it. This process is aimed at verifying whether the candidate is an Extract Method refactor or not. Specifically, the candidate will be checked for any signs of useless patterns of refactoring as outlined in [30], including directly deleting the clone, incomplete refactoring, refactoring on non-target clone, and helper method extraction without replacing the clone occurrences. When the LLM-based decision is unavailable or rejects the candidate, the system further applies PSI-based structural analysis. For whole-method clones, it checks whether the original target methods are preserved and calls a newly extracted helper. For fragment clones, it checks whether the duplicated fragments are actually replaced or substantially reduced. Only candidates that pass this usefulness check proceed to compilation and testing.

The Refactoring Agent is also guided by retrieval-augmented generation (RAG). The retrieved examples come from real Extract Method refactorings. If a refactoring fails the usefulness check, it is rejected, and feedback is given to the refactoring agent. This allows the workflow to gradually move toward refactorings that are both correct and meaningful.

Accepted refactoring. A refactoring is considered *accepted* if it passes all stages of the workflow and is finally written back to the original source file. At this point, the refactoring has satisfied the usefulness criteria, successfully compiled, passed all tests, and received approval from the human developer.

4.2. Detection Agent

Figure 2 presents the overall workflow of the Detection Agent. The process is triggered when there is code snippet copied (1). Instead of scanning the entire project, the workflow restricts the analysis to the updated source file containing

the pasted code (2). This design reduces token consumption and enables interactive clone detection close to the moment when duplication is introduced. The pasted snippet, together with the full source file, is then forwarded to three independent LLM-based detection panelists (3). Each panelist analyzes the same input separately and produces a list of candidate clone groups that are structurally similar to the pasted snippet (4–6). To improve robustness, the prompts also include retrieved few-shot examples from the clone database and require the returned clone snippets to remain exact substrings of the original source code.

After the panelists generate their outputs, a Detection Curator votes and evaluates the candidate clone groups to produce the final detection result (7). The curator aggregates the panelist outputs using majority voting and ensures that all reported clones are associated with the pasted snippet. The resulting clone groups are then grounded back to the source file to correct inaccurate LLM-generated line ranges and merge duplicate intervals (10–14). If the LLM-based detection fails to identify any valid clones, the workflow falls back to a lightweight PSI-based detector that incrementally searches for Type-1, Type-2, and conservative Type-3 clones using normalized token matching and similarity comparison (16–19). The final output of the Detection Agent is a structured clone representation that can later be reviewed and manually selected by developers before being forwarded to the Refactoring Agent (15).

Listing 2: Detection Agent panelist prompt.

```
You are one of three independent clone-detection panelists.
Analyze the following Java source code from a single file named '<
FILE_NAME>'.
Your task is to exhaustively find every non-trivial clone of the user
pasted snippet within this same file only.

You MUST focus on the pasted snippet and enumerate all other code regions
in the file that are clones of it.
Do NOT report clones that do not involve the pasted snippet.
Ignore only truly trivial single-line repetitions.
Do not reject short, method-level, symmetric, or small-substitution
clones merely because only a few identifiers, literals, casts,
method names, or API calls differ.
Do not stop after finding the first match. Keep searching until you have
listed all relevant matches involving the pasted snippet.

The pasted snippet is located in this file around lines <
SNIPPET_START_LINE> to <SNIPPET_END_LINE>.
If the exact line location is unknown, locate the pasted snippet
occurrence in the file yourself.

=== Few-shot examples (from clone database) ===
<FEW_SHOT_EXAMPLES>
=== End few-shot examples ===
```

```

File source:
'''
<FILE_SOURCE>
'''

Selected snippet:
'''
<SNIPPET_PASTED>
'''

IMPORTANT CONSTRAINTS:
1) Every clone you report MUST involve the pasted snippet occurrence in
   this file.
2) Return ALL non-trivial matching occurrences related to the pasted
   snippet, not just one example match.
3) In "ranges", include every matching range for that clone class in this
   file. One range SHOULD correspond to the pasted snippet occurrence,
   and all other matching ranges elsewhere in the file SHOULD also be
   included.
4) If there are multiple distinct clone classes involving the pasted
   snippet, return multiple objects in "clones".
5) Focus on structural similarity, even when a few identifiers, literals,
   casts, method names, or API calls differ.
6) Return cloneCodes as an array of verbatim code snippets aligned with
   ranges. cloneCodes[i] must correspond to ranges[i].
7) Prefer returning the pasted snippet occurrence as cloneCodeA and one
   representative matching region as cloneCodeB, but still include ALL
   occurrence snippets in cloneCodes.
8) Do NOT rewrite, reformat, rename variables, or add missing context
   when copying cloneCodes from fileSource.
9) If no reasonably similar code is found, output status "no_clones" with
   an empty clones list.

Output ONLY a valid JSON object with the following structure:
{
  "status": "found_clones" or "no_clones",
  "file": "<FILE_NAME>",
  "clones": [
    {
      "id": "unique_clone_id",
      "ranges": [
        { "startLine": int, "endLine": int },
        ...
      ],
      "cloneCodes": [
        "verbatim_occurrence_code_aligned_with_ranges[0]",
        "verbatim_occurrence_code_aligned_with_ranges[1]",
        ...
      ],
      "refactorType": "extracted_method" or "extracted_class" or other
    }
  ]
}

```

```

    string,
    "reason": explanation of why this clone was detected and why all
    listed ranges belong to the same clone class,
    "cloneCodeA": pasted-snippet occurrence copied verbatim from
    fileSource when possible,
    "cloneCodeB": matching clone region copied verbatim from fileSource
    when possible
  },
  ...
]
}

```

Do not include any text outside the JSON object.

Listing 3: [Detection Curator prompt](#).

You are the detection curator.
 You must review three panelist outputs and produce the **final** clone detection result.
 Apply a majority vote across the three panelists.

- If 2 or more panelists provide credible evidence of a clone, **return** found_clones.
- If 2 or more panelists report no clones or provide weak/unparsed evidence, **return** no_clones.
- If exactly one panelist finds a clone, only **return** found_clones **if** that panelist evidence is exceptionally strong: precise line ranges and clear structural similarity. Otherwise **return** no_clones.

Merge duplicate or overlapping clone groups when they clearly refer to the same pasted-snippet clone **class**.
 Return ONLY one JSON object and no extra text.

Analyze the following Java source code from a single file named '<
 FILE_NAME>'.
 Your task is to exhaustively find every non-trivial clone of the user pasted snippet within **this** same file only.
 You MUST focus on the pasted snippet and enumerate all other code regions in the file that are clones of it.
 Do NOT report clones that **do** not involve the pasted snippet.
 Ignore only truly trivial single-line repetitions.
 Do not reject **short**, method-level, symmetric, or small-substitution clones merely because only a few identifiers, literals, casts, or method names differ.
 Do not stop after finding the first match. Keep searching until you have listed all relevant matches involving the pasted snippet.

The pasted snippet is located in **this** file around lines <
 SNIPPET_START_LINE> to <SNIPPET_END_LINE>.
 If the exact line location is unknown, locate the pasted snippet occurrence in the file yourself.

File source:

```

'''
<FILE_SOURCE>
'''

Selected snippet:
'''
<SNIPPET_PASTED>
'''

IMPORTANT CONSTRAINTS:
1) Every clone you report MUST involve the pasted snippet occurrence in
   this file.
2) Return ALL non-trivial matching occurrences related to the pasted
   snippet, not just one example match.
3) In "ranges", include every matching range for that clone class in this
   file. One range SHOULD overlap the pasted snippet approximate
   location, and all other matching ranges elsewhere in the file SHOULD
   also be included.
4) If there are multiple distinct clone classes involving the pasted
   snippet, return multiple objects in "clones".
5) Focus on structural similarity, even when a few identifiers, literals,
   casts, method names, or API calls differ.
6) Return cloneCodes as an array of verbatim code snippets aligned with
   ranges. cloneCodes[i] must correspond to ranges[i].
7) Prefer returning the pasted snippet occurrence as cloneCodeA and one
   representative matching region as cloneCodeB, but still include ALL
   occurrence snippets in cloneCodes.
8) cloneCodes entries do not need to be textually identical. Small
   substitutions still count as a valid clone.
9) If no reasonably similar code is found, output status "no_clones" with
   an empty clones list.

IMPORTANT: For each detected clone, include cloneCodes for ALL listed
   ranges as verbatim copies from the provided fileSource when possible
   .
- startLine/endLine are 1-based inclusive line numbers in fileSource, and
  each range must tightly cover its matching cloneCodes entry.
- If a cloneCodes entry and its line numbers disagree, prioritize making
  cloneCodes verbatim from fileSource and make the line numbers match
  that exact text.
- Do NOT rewrite, reformat, rename variables, or add missing context when
  copying cloneCodes from fileSource.
- A clone may still be valid even when the two regions differ slightly in
  identifiers, literals, casts, or one method call.
- A single clone object may have more than two ranges, and if so you
  should include all of them.
- If you can identify three or more matching occurrences of the same
  pasted snippet clone class in this file, all of them must appear in
  "ranges".

```



```

=== PANELIST OUTPUTS ===
[P1]
<PANELIST_1_OUTPUT>

[P2]
<PANELIST_2_OUTPUT>

[P3]
<PANELIST_3_OUTPUT>

Rules:
- The final JSON must follow the exact schema below.
- Keep only clone groups that involve the pasted snippet occurrence in this file.
- Preserve all known matching ranges for a clone group when the evidence is strong enough.
- Use no_clones with an empty clones list when the panelists do not provide convincing clone evidence.

Output ONLY a valid JSON object with the following structure:
{
  "status": "found_clones" or "no_clones",
  "file": "<FILE_NAME>",
  "clones": [
    {
      "id": "unique_clone_id",
      "ranges": [
        { "startLine": int, "endLine": int },
        ...
      ],
      "cloneCodes": [
        "verbatim_occurrence_code_aligned_with_ranges[0]",
        "verbatim_occurrence_code_aligned_with_ranges[1]",
        ...
      ],
      "refactorType": "extracted_method" or "extracted_class" or other string",
      "reason": "explanation of why this clone was detected and why all listed ranges belong to the same clone class",
      "cloneCodeA": "pasted-snippet occurrence copied verbatim from fileSource when possible",
      "cloneCodeB": "matching clone region copied verbatim from fileSource when possible"
    },
    ...
  ]
}
Do not include any text outside the JSON object.

```

Listings 2 and 3 show the prompts used by the Detection Agent. The detec-

tion process contains two prompt types: one for the independent panelists and one for the curator. These prompts contain the following components:

- A task description instructing the model to identify cloned code fragments related to the pasted snippet within a single Java file.
- The full source code of the target file and the pasted snippet.
- Few-shot examples retrieved from a clone RAG database for the panelist prompt.
- Three independent panelist outputs for the curator prompt.
- Explicit constraints requiring all reported clones to involve the pasted snippet and requiring copied clone snippets to be exact substrings of the input code.
- A structured JSON output format specifying clone locations, aligned clone code snippets, refactoring type, and detection reasons.

We are including these elements to make the detection process more reliable and controllable. By giving the entire source file, we ensure that the model has enough information and will not assume any missing lines of code. The few-shot examples train the model on how to detect realistic clone pairs. The substring condition is important because it makes sure that the clones detected were copied directly from the source file. This is necessary when performing the refactoring process.

The output of the Detection Agent is a structured clone description. After this step, users can manually select which detected clones they want to refactor. This design introduces a human-in-the-loop step, allowing developers to focus on relevant or high-priority clones and avoid unnecessary transformations. The selected clone instances are then forwarded to the Refactoring Agent as input.

4.2.1. Algorithm: Detection Agent

Algorithm 1 and Figure 2 describes the procedure implemented by the Detection Agent to identify clone candidates within a single Java file. As shown in lines 3–5, the agent first checks whether the pasted snippet is empty and directly returns `no_clones` if so. Otherwise, as shown in line 6, the agent localizes the pasted snippet within the file and records its approximate line range (l_s, l_e) for subsequent grounding and filtering.

The Detection Agent then invokes three independent LLM detection panelists (lines 7–12). Each panelist receives a prompt containing the full file source, the pasted snippet, optional few-shot examples retrieved from the clone database, and instructions to report only clone groups involving the pasted snippet, as shown in Listing 2. Each panelist returns a JSON detection result, which is parsed and normalized by grounding the reported clone ranges against the actual source text.

After collecting the panelist outputs, a detection curator is invoked (lines 14–21) to consolidate overlapping clone groups, filter out unconvincing evidence, and produce the final detection result. If the curator output is valid, it is normalized; otherwise, the system falls back to merging the valid panelist outputs.

The normalization process is essential because LLM-reported line numbers are often inaccurate. The framework uses the detected clone codes to locate exact or trimmed matches in the source file. When multiple candidate locations exist, the system selects the most appropriate one by penalizing repeated range usage and non-exact matches while favoring overlap with the pasted snippet.

If the LLM-based detection yields no clones, the workflow falls back to a lightweight PSI-based approach (lines 22–24). This fallback proceeds in stages. It first searches for repeated instances of the inserted fragment using conservative text comparison. If exact text matching fails, it performs Type-1 token comparison by ignoring whitespace and comments. It then extends the search to Type-2 clones by normalizing identifiers and literals before requiring an exact normalized token-sequence match. Finally, as a conservative Type-3 fallback, it applies approximate normalized-token matching over bounded token windows using an LCS-based similarity threshold, allowing small insertions or deletions while limiting false positives.

Finally, the workflow resolves clone intervals, merges overlapping groups, assigns the status `found_clones`, and returns the detection result (lines 28–32), which is then forwarded to the Refactoring Agent for Extract Method candidates.

Algorithm 1 [Detection Agent](#)

```
1: Input: project context proj, file name f, file source S, pasted snippet Ssnip,  
   LLM caller  $\mathcal{L}(\cdot)$   
2: Output: detection result R with status, file, and clones  
3: if Ssnip is empty then  
4:   return {status = no_clones, file = f, clones = []}  
5: end if  
6:  $(l_s, l_e) \leftarrow \text{FINDSNIPPETLINERANGE}(S, S_{snip}) \triangleright$  best-effort snippet grounding  
7: PanelistResults  $\leftarrow []$   
8: for  $p \in \{P1, P2, P3\}$  do  
9:    $P_p \leftarrow \text{BUILDPANELISTDETECTIONPROMPT}(p, proj, S, S_{snip}, f, l_s, l_e)$   $\triangleright$   
   may include retrieved few-shot examples  
10:   $raw_p \leftarrow \mathcal{L}(P_p)$   
11:   $R_p \leftarrow \text{PARSEDETECTIONRESULT}(raw_p, f)$   
12:   $\hat{R}_p \leftarrow \text{NORMALIZEDETECTIONRESULT}(R_p, f, S, S_{snip}, l_s, l_e)$   
13:  PanelistResults.ADD(p,  $raw_p$ ,  $R_p$ ,  $\hat{R}_p$ )  
14: end for  
15:  $P_c \leftarrow \text{BUILDETECTIONCURATORPROMPT}(f, S, S_{snip}, l_s, l_e, \text{PanelistResults})$   
16:  $raw_c \leftarrow \mathcal{L}(P_c)$   
17:  $R_c \leftarrow \text{PARSEDETECTIONRESULT}(raw_c, f)$   
18: if  $R_c \neq \text{null}$  then  
19:    $R \leftarrow \text{NORMALIZEDETECTIONRESULT}(R_c, f, S, S_{snip}, l_s, l_e)$   
20: else  
21:    $R \leftarrow \text{MERGEPANELISTRESULTS}(\text{PanelistResults}, f)$   
22: end if  
23: if  $R = \text{null}$  or  $R.\text{clones}$  is empty then  
24:    $R \leftarrow \text{PSIFALLBACKDETECT}(proj, S, S_{snip}, f)$   
25: end if  
26: if  $R = \text{null}$  or  $R.\text{clones}$  is empty then  
27:   return {status = no_clones, file = f, clones = []}  
28: end if  
29:  $R.\text{clones} \leftarrow \text{RESOLVECLONERANGESWITHPSI}(proj, f, S, R.\text{clones})$   
30:  $R.\text{clones} \leftarrow \text{MERGEOVERLAPPINGCLONGROUPS}(S, R.\text{clones})$   
31:  $R.\text{status} \leftarrow \text{found\_clones}$   
32:  $R.\text{file} \leftarrow f$   
33: return R
```

4.3. Refactoring Agent

The Refactoring Agent generates an Extract Method refactoring candidate based on the clone selected by the Detection Agent. In this study, the agent is strictly limited to a single refactoring type: *Extract Method*. Other refactoring types are explicitly disallowed.

Figure 3 illustrates the workflow of the Refactoring Agent after clone candidates are produced by the Detection Agent (1). The process begins when the user selects a target clone group (2). The workflow then prepares the full source

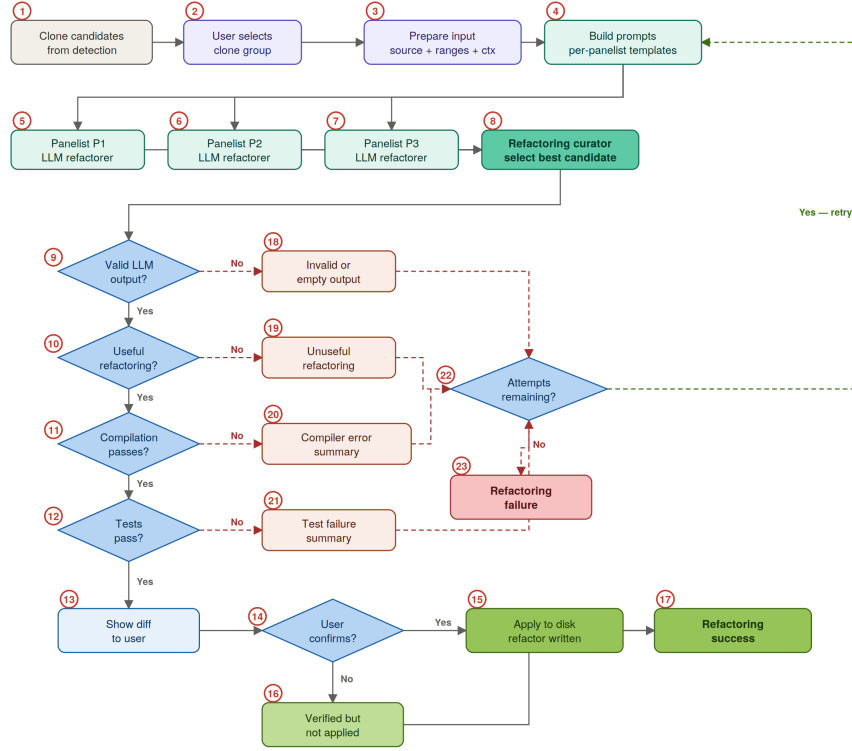


Figure 3: Refactoring Pipeline.

file, clone ranges, and contextual information required for refactoring generation (3). Based on this information, the system constructs panelist-specific prompts for the three independent refactoring panelists (4). Each panelist independently generates an Extract Method candidate for the selected clone (5–7). The generated candidates are then evaluated by the Refactoring Curator, which selects the most suitable candidate for downstream validation (8).

The selected candidate first undergoes validation to determine whether the generated LLM output is structurally valid (9). The workflow then evaluates whether the candidate represents a useful Extract Method refactoring according to the predefined usefulness criteria (10). Candidates that pass usefulness validation proceed to compilation validation (11) and testing validation (12). If all validation stages succeed, the workflow presents the resulting code differences to the user (13). The user may then decide whether to apply the refactoring (14). Accepted refactorings are written to disk and applied to the editor (15), leading to successful workflow completion (17). If the user rejects the modification, the candidate is considered verified but not applied (16). If any validation stage

fails, the workflow generates summarized failure feedback, including invalid or empty outputs (18), unuseful refactorings (19), compilation failures (20), or testing failures (21). The workflow then determines whether additional retry attempts remain (22). If no remaining attempts are available, the workflow terminates with refactoring failure (23).

Listing 4: Refactoring Agent panelist prompt with RAG context.

```
You are one of three independent Java refactoring panelists.
Produce your own best Extract Method candidate for the provided target
clone.
Do not try to match the other panelists. Return the strongest valid
candidate you can.

You are an expert Java refactoring agent. You specialize in clone removal
via Extract Method and improving software quality. Follow the
constraints and output format EXACTLY. Do not skip the refactoring
task. If the target clone can be extracted with parameters, you must
still perform Extract Method. If anything is unclear, prefer the
most conservative valid Extract Method that targets the provided
clone rather than leaving the file unchanged.

=== WORKING SET (Single File Only) ===
File name: <FILE_NAME>
You are given the FULL file source below. You may ONLY modify this file.

<FILE_SOURCE_BEGIN>
<FILE_SOURCE>
<FILE_SOURCE_END>

=== CONTEXT ===
Language: Java
Goal: Remove duplicated code with Extract Method.
Constraints: Preserve public API, preserve package/imports, and preserve
semantics.

=== CLONE CONTEXT ===
Clone ID: <CLONE_ID>
Approximate target clone ranges:
<CLONE_RANGES>

Representative clone snippet:
<REPRESENTATIVE_CLONE_SNIPPET>

=== FEW-SHOT GUIDANCE (RAG) ===
Use these examples only as guidance for structure and style. Do NOT copy
verbatim.
<OPTIONAL_RAG_EXAMPLES>

=== REFACTORING TASK ===
```

Refactoring Pattern: Extract Method (ONLY)

Intent: Improve maintainability and reduce duplication `while` preserving behavior.

Requirements:

- 1) Refactor the provided target clone, not some other duplication in the file.
- 2) Extract the duplicated logic into one `private` helper method.
- 3) Replace ALL target clone occurrences with calls to the extracted method.
- 4) Prefer a parameterized helper method when the clone occurrences have small local differences.
- 5) Choose the smallest safe extraction boundary that still removes the target duplication.
- 6) Ensure the result compiles as valid Java.
- 7) Do not leave the target clone unrefactored unless Extract Method is truly impossible within `this` file.

=== STRICT CONSTRAINTS ===

- Modify ONLY `this` file.
- Preserve the `package` line and ALL `import` statements EXACTLY.
- Do NOT change `public/protected` method signatures or `class public` API.
- Do NOT introduce `new` classes or `new` files.
- The provided clone snippet is the mandatory refactoring target.
- The clone ranges are approximate hints and must not override the snippet.
- Do NOT refactor unrelated duplicated code elsewhere in the file.
- Do NOT change external behavior.
- Minimize edits outside the target clone regions.
- Do NOT add explanatory prose outside the JSON output.

=== TARGET OCCURRENCES ===

You must refactor ALL of the following occurrences.

<TARGET_OCCURRENCES_BEGIN>

OCCURRENCE_1:

<CLONE_OCCURRENCE_1>

OCCURRENCE_2:

<CLONE_OCCURRENCE_2>

<TARGET_OCCURRENCES_END>

=== OUTPUT FORMAT ===

Output ONLY a valid JSON object. Do NOT `return` the whole file. Do NOT use markdown fences.

```
{
  "helper_method": full private helper method declaration only,
  "occurrence_replacements": [
    {
```

```

    "occurrence_id": "OCCURRENCE_1",
    "replacement_code": only the code that should replace that
    occurrence
  },
  {
    "occurrence_id": "OCCURRENCE_2",
    "replacement_code": only the code that should replace that
    occurrence
  }
]
}

```

The `helper_method` must contain only the extracted helper method, not the `class` or file. Each `replacement_code` must contain only the replacement `for` that occurrence. Include one replacement entry `for` every listed occurrence. Do not include explanation text before or after the JSON.

Listing 4 shows the prompt for the Refactoring Agent panelists. Inspired by previous work [15, 23, 8, 19], the refactoring prompt consists of:

- A role instruction describing the agent as a Java refactoring expert.
- A hard constraint specifying that only Extract Method refactoring is allowed.
- The full source code of the current file.
- The selected clone ranges, representative clone snippet, and target clone occurrences.
- Retrieved examples of Extract Method refactorings.
- Feedback from previous usefulness, compilation, or testing failures, if available.
- An output instruction requiring a structured JSON refactoring plan rather than a full rewritten file.

Each prompt component serves a specific purpose. The role instruction encourages the model to reason as a Java refactoring expert. The Extract Method constraint prevents the model from producing unrelated transformations such as deleting clone code, merging methods, or applying a different refactoring. The full source file provides the surrounding context needed to preserve dependencies, visibility, imports, and method signatures. The clone snippets and occurrence identifiers focus the model on the detected target clone rather than unrelated duplicated code elsewhere in the file. Retrieved examples provide guidance on the structure of valid Extract Method transformations without being copied directly. Feedback from previous usefulness, compilation, or testing failures allows the model to revise unsuccessful candidates across attempts. Finally, requiring a structured JSON plan reduces the risk of uncontrolled full-file

rewrites: the model proposes only the helper method and clone replacements, while the system reconstructs the complete candidate source code deterministically before validation.

Listing 5: Refactoring Curator prompt.

```
You are the refactoring curator.
You must review three candidate Extract Method refactorings.
Select the single best useful panelist candidate for downstream
usefulness validation.
A useful panelist candidate must clearly achieve EXTRACT_METHOD_CONFIRMED
and must not trigger any not-useful category.

=== USEFUL CATEGORY DEFINITION ===
- EXTRACT_METHOD_CONFIRMED: The candidate clearly extracts one helper and
  replaces all target clone occurrences with calls to it.

=== NOT-USEFUL CATEGORY DEFINITIONS ===
- EXTRACT_METHOD_NOT_FOUND: No valid Extract Method is visible for the
  target clone.
- INCOMPLETE_REFACTORING_DETECTED: Some duplicated logic still remains in
  the original target methods after refactoring.
- EXTRACTION_WITHOUT_CLONE_REPLACEMENT_DETECTED: A helper was introduced,
  but the original duplicate body still remains instead of being
  replaced.
- DIRECT_CLONE_REMOVAL_DETECTED: A clone occurrence was deleted directly
  instead of extracting shared logic.
- POST_EXTRACTION_CLONE_DELETION_DETECTED: A helper was added, but one
  target clone was deleted afterward.
- CALL_BASED_CLONE_SUBSTITUTION_DETECTED: One target clone now calls
  another existing method instead of using a new shared helper.
- CLONE_REMOVAL_BY_DELEGATION_DETECTED: The candidate relies on
  delegation without clearly extracting one shared helper for all
  target clones.
- FRAGMENTATION_OF_LOGIC_DETECTED: Shared logic is split across multiple
  helpers instead of one coherent extraction.
- NON_TARGET_CLONE_REFACTORING_DETECTED: The candidate seems to refactor
  some other duplication while the intended target remains unresolved.
- EXCESSIVE_REFACTORING_DETECTED: The candidate pulls in code beyond the
  intended cloned fragment.

=== TARGET FILE ===
File name: <FILE_NAME>
Full file source (for context only):
<FILE_SOURCE>

=== TARGET CLONE ===
Clone ID: <CLONE_ID>
Requested refactor type: <REFACTOR_TYPE>
Detection rationale: <DETECTION_REASON>
```

```

OCCURRENCE_1:
<CLONE_OCCURRENCE_1>

OCCURRENCE_2:
<CLONE_OCCURRENCE_2>

=== PANELIST CANDIDATES ===
[P1]
{
  "panelist_id": "P1",
  "parsed": <P1_PARSED>,
  "status": "<P1_STATUS>",
  "message": "<P1_MESSAGE>",
  "error": "<P1_ERROR>",
  "helper_method": "<P1_HELPER_METHOD>",
  "occurrence_replacements": <P1_OCCURRENCE_REPLACEMENTS>,
  "candidate_source_preview": "<P1_CANDIDATE_SOURCE_PREVIEW>",
  "raw_response_preview": "<P1_RAW_RESPONSE_PREVIEW>"
}

[P2]
{
  "panelist_id": "P2",
  "parsed": <P2_PARSED>,
  "status": "<P2_STATUS>",
  "message": "<P2_MESSAGE>",
  "error": "<P2_ERROR>",
  "helper_method": "<P2_HELPER_METHOD>",
  "occurrence_replacements": <P2_OCCURRENCE_REPLACEMENTS>,
  "candidate_source_preview": "<P2_CANDIDATE_SOURCE_PREVIEW>",
  "raw_response_preview": "<P2_RAW_RESPONSE_PREVIEW>"
}

[P3]
{
  "panelist_id": "P3",
  "parsed": <P3_PARSED>,
  "status": "<P3_STATUS>",
  "message": "<P3_MESSAGE>",
  "error": "<P3_ERROR>",
  "helper_method": "<P3_HELPER_METHOD>",
  "occurrence_replacements": <P3_OCCURRENCE_REPLACEMENTS>,
  "candidate_source_preview": "<P3_CANDIDATE_SOURCE_PREVIEW>",
  "raw_response_preview": "<P3_RAW_RESPONSE_PREVIEW>"
}

=== SELECTION RULES ===
1) First classify every panelist candidate using the useful and not-
    useful definitions above.

```

```

2) A panelist candidate is useful only if it clearly triggers
   EXTRACT_METHOD_CONFIRMED and triggers none of the not-useful
   categories.
3) Reject panelist candidates that likely trigger any not-useful category
   : EXTRACT_METHOD_NOT_FOUND, INCOMPLETE_REFACTORING_DETECTED,
   EXTRACTION_WITHOUT_CLONE_REPLACEMENT_DETECTED,
   DIRECT_CLONE_REMOVAL_DETECTED,
   POST_EXTRACTION_CLONE_DELETION_DETECTED,
   CALL_BASED_CLONE_SUBSTITUTION_DETECTED,
   CLONE_REMOVAL_BY_DELEGATION_DETECTED,
   FRAGMENTATION_OF_LOGIC_DETECTED,
   NON_TARGET_CLONE_REFACTORING_DETECTED,
   EXCESSIVE_REFACTORING_DETECTED.
4) If one or more panelist candidates are useful, choose the best useful
   candidate. Prefer the most conservative change with the smallest
   safe extraction boundary.
5) If no panelist candidate is useful, still choose the least-bad valid
   panelist candidate; do not generate your own refactoring plan.
6) If a panelist candidate failed to parse or apply, treat it as not
   useful.

=== OUTPUT FORMAT ===
Return ONLY a JSON object with this exact shape.
{
  "decision": "select_panelist",
  "selected_panelist_id": "P1",
  "matched_categories": ["EXTRACT_METHOD_CONFIRMED"],
  "summary": "short explanation of why this candidate is best, or why no
    candidate is useful",
  "feedback": "optional short guidance about the remaining risk of the
    selected candidate; otherwise empty string",
  "confidence": 0.85
}

```

Listing 5 is the prompt for Refactoring Curator. In designing the Refactoring Curator prompt, we want to ensure that the selection of candidates is clear and repeatable in future tries. Because the Refactoring Agent creates three candidates independently, the curator doesn't create any new code; rather, it takes the three panelists' results and picks the one candidate that is more likely to pass the usefulness checker. With this understanding in mind, our prompt for Refactoring Curator includes the target file, the chosen clone context, the parsed status of the candidates, the helper method, occurrence replacements, and the resulting source code.

The prompt also contains useful category advice to ensure consistency between the curator's choice and the criteria for validating this choice in a subsequent step of the process. Instead of choosing the candidate whose code seems most plausible on first impression, the curator is encouraged to favor candidates that can be seen to reliably extract one common helper function and then replace all target code instances, but which avoid the unuseful patterns of deleting a

clone directly, partial refactoring, helper extraction without replacing all target clones, and refactoring of non-target clones. The requirement that the curator provide a structured JSON file enables automated tracking of the panelist’s choice, reasoning, matching of categories, feedback, and confidence level.

4.4. Compilation Validator

Following the design idea of RefAgent [27], we incorporate a Compilation Validator, which is responsible for validating the syntactic correctness of the generated refactoring code.

Specifically, the validator writes the refactored source code to a temporary directory and invokes the Java compiler using the project’s original classpath. Only compilation errors related to the modified file are considered.

The compilation process follows these steps:

- Create a temporary source directory that mirrors the original package structure.
- Write the refactored source code into this directory.
- Invoke the Java compiler with the project classpath.
- Capture and summarize compiler error messages, if any.

If compilation fails, the error summary is sent back to the Refactoring Agent as feedback. If compilation succeeds, the workflow proceeds to the Testing validator.

4.5. Testing Validator

The Testing Validator ensures that the refactored code preserves the original program behavior. The agent uses EvoSuite [18] to generate unit tests automatically, and then it executes the test files. To ensure that tests exercise the refactored code, the compiled bytecode from the temporary compilation output is placed before the original classes on the classpath. Automatically generated tests do not always fully cover the cloned methods or all relevant execution paths. Among the 120 clone cases in our study, 79 cases have test coverage that directly exercises the cloned methods. For the remaining 41 cases, the generated tests do not directly invoke the cloned fragments. In these situations, we use GPT-5.5 to generate tests and manually run the code to verify that the refactoring preserves behavior.

The Testing Validator operates using the following information:

- The fully qualified name of the target class.
- A summary of the refactoring that was applied.
- The refactored source code.
- Execution logs from previous test runs, if available.

If all tests pass, we consider the refactoring preserves the original functionality. If tests fail, a summarized test failure report is generated and sent back to the Refactoring Agent. After a number of iterations defined by developers, refactorings that still fail the testing are discarded. This setup guarantees that all tests are executed against the proposed refactoring.

4.5.1. Algorithm: Refactoring with Compilation and Testing Gates

Algorithm 2, Algorithm 3 and Figure 3 presents the workflow that is triggered after clone detection. The purpose of this stage is to iteratively generate, validate, and optionally apply an Extract Method refactoring while preserving program behavior.

The process begins by setting up an initially empty feedback message and a retry mode flag. The feedback message holds data from past failures, whereas the retry mode flag decides if future refactorings will need a revision prompt with feedback only. In addition, the workflow captures snapshots of the source code for methods that hold the selected clone occurrences. These snapshots enable us to know whether the user changes the methods manually during the workflow’s execution.

The algorithm proceeds to an iteration loop. The number of iterations K depends on user preferences. The start of every iteration cycle is preceded by checking for cancellation or a change of any monitored clone operation method by the user. In case of positive results from this check, the workflow is canceled, because its continuation would result in refactoring of outdated code.

In each iteration, the Refactoring Agent produces an Extract Method refactoring transformation candidate. If the previous failure resulted from the Usefulness Checker, the Refactoring Agent receives a feedback-only prompt. Otherwise, the prompt consists of the current source code file, the selected clone, potentially found examples of refactoring, and feedback based on previous failures. Internally, the Refactoring Agent relies on three independent panelists for generating candidates and a curator for choosing the best among the candidates.

Unlike the strategy of returning a whole-file rewriting, the Refactoring Agent is programmed to generate a partially-refactored solution containing the extracted method along with the replacement code. Next, the partially-refactored solution is applied to the original source code by identifying all occurrences of the clones, replacing these with the appropriate helper calls or replacement code, and embedding the helper method in the original enclosing class. If no suitable candidate source code can be found or created, the process will be stopped, and feedback provided.

Prior to compilation and testing, the candidate undergoes assessment through the Usefulness Checker module. In this phase, the candidate will be determined as being useful for the Extract Method or deemed as a useless transformation. To determine its usefulness, the checker will perform an initial LLM panel-and-curator evaluation on the focused before-and-after code contexts. When the decision from the LLM cannot be obtained, or the candidate transformation is not approved by the LLM, then the candidate will undergo structural analysis using PSI. If the candidate is determined to be a whole-method clone, the

checker will validate that the original target methods remain unchanged but will delegate or invoke a newly extracted helper method. On the other hand, if the candidate is a fragment clone, it will confirm that the code fragments have been replaced or significantly reduced.

Once the check for usefulness is successful, the next phase of the workflow is the isolated compilation step. Here, the candidate source is compiled into an output directory, which is done without making any changes to the original source code. The process is based on the project's classpath and the source roots. If there are any compilation errors, the errors may be compared to the compilation errors generated when compiling the original source.

After successful compilation, the process moves on to testing. The system determines the fully qualified name of the primary class from the proposed input and then uses the generated patches from the isolated compilation process in setting up the test runner. This guarantees that the tests are run using the transformed code base instead of the original code base. In case the class under transformation could not be located, or if there was a failure during testing, a summary is written out as feedback.

After passing the stages of usefulness checking, compilation, and testing, the process shows the diff between the source that is currently present and the source that would result from applying the refactor. At this point, the user makes a decision to either accept or reject the refactoring. In case the user accepts the refactoring, it is saved to the source file, and the process ends successfully.

In case no candidate clears all the validation gates in K trials, failure is indicated by the workflow. This shows how usefulness test, compilation, and testing act as sequential gates for the guidance of the Refactoring Agent to achieve an Extract Method refactoring that preserves the functionality of the code.

Algorithm 2 Refactoring Workflow: Generation and Usefulness Gate

```
1: Input: project context  $proj$ , file name  $f$ , current source  $S_{cur}$ , detected clone  $c$ , RAG  
   guidance  $E_{rag}$ , max attempts  $K$ , LLM caller  $\mathcal{L}(\cdot)$   
2: Output: candidate refactoring  $S_{cand}$  and feedback state, or failure  
3:  $feedback \leftarrow$  empty  
4:  $useFeedbackOnly \leftarrow false$   
5:  $watchedMethods \leftarrow \text{CAPTURECLONEMETHODSNAPSHOTS}(proj, f, c)$   
6: for  $attempt = 1$  to  $K$  do  
7:   if  $\text{WORKFLOWCANCELLED}$  then  
8:     return failure  
9:   end if  
10:  if  $\text{CLONEMETHODMODIFIED}(proj, f, watchedMethods)$  then  
11:     $\text{CANCELWORKFLOW}$   
12:    return failure  
13:  end if  
14:  if  $useFeedbackOnly$  and  $feedback$  is not empty then  
15:     $RR \leftarrow \text{REFACTORWITHFEEDBACKPROMPT}(f, S_{cur}, c, feedback, \mathcal{L})$   
16:  else  
17:     $fb \leftarrow \text{concat}(E_{rag}, feedback)$   
18:     $RR \leftarrow \text{REFACTORWITHPANELISTSANDCURATOR}(f, S_{cur}, c, fb, \mathcal{L})$   
19:  end if  
20:  if  $RR = \text{null}$  or  $RR.\text{newSource}$  is empty then  
21:     $feedback \leftarrow$  "Refactor produced empty or invalid output."  
22:     $useFeedbackOnly \leftarrow false$   
23:    continue  
24:  end if  
25:   $S_{cand} \leftarrow RR.\text{newSource}$   
26:   $\text{SAVEPROPOSALFORINSPECTION}(proj, f, S_{cand}, attempt)$   
27:   $UR \leftarrow \text{USEFULNESSCHECK}(proj, f, c, S_{cur}, S_{cand}, watchedMethods, \mathcal{L})$   
28:  if  $UR.\text{isUseful} = false$  then  
29:     $feedback \leftarrow \text{BUILDUSEFULNESSFEEDBACK}(UR, S_{cand})$   
30:     $useFeedbackOnly \leftarrow true$   
31:    continue  
32:  end if  
33:  return  $(S_{cand}, feedback, useFeedbackOnly, watchedMethods)$   
34: end for  
35: return failure
```

4.6. Usefulness Checker

Even if we explicitly instruct the LLM to apply the Extract Method only, the generated refactoring is often not a proper Extract Method in practice. A previous study [30] observed that LLMs frequently produce refactorings that look reasonable at the surface level but do not actually remove duplication as Extract Method normally does. These outcomes include *incomplete refactoring*, *post-extraction clone deletion*, *direct clone removal*, *call-based clone substitution*, *clone removal by delegation*, and *fragmentation of logic*.

In this work, our goal is to operationalize these observations into concrete and reproducible checks that can be executed automatically. Therefore, we introduce a usefulness checking stage that runs immediately after refactoring and analyzes the code before and after the change.

Figure 4 illustrates the workflow of the Usefulness Checker. The process begins when the candidate refactored file produced by the Refactoring Agent is

Algorithm 3 Refactoring Workflow: Compilation, Testing, and Application Gates

```

1: Input: project context  $proj$ , file name  $f$ , current source  $S_{cur}$ , candidate source  $S_{cand}$ ,
   feedback state, watched methods
2: Output: success (applied or verified) or failure
3: if CLONEMETHODMODIFIED( $proj, f, watchedMethods$ ) then
4:   CANCELWORKFLOW
5:   return failure
6: end if
7:  $log_{comp} \leftarrow \text{COMPILEPROPOSEDSOURCETOTEMP}(proj, f, S_{cand})$ 
8:  $CR \leftarrow \text{ANALYZECOMPILELOG}(f, log_{comp})$ 
9: if  $CR.status \neq compile\_ok$  then
10:   $CR \leftarrow \text{IGNOREPREEXISTINGBASELINEERRORS}(CR, S_{cur})$ 
11: end if
12: if  $CR.status \neq compile\_ok$  then
13:   $feedback \leftarrow CR.summary$ 
14:   $useFeedbackOnly \leftarrow false$ 
15:  return failure
16: end if
17: if CLONEMETHODMODIFIED( $proj, f, watchedMethods$ ) then
18:  CANCELWORKFLOW
19:  return failure
20: end if
21:  $targetFqn \leftarrow \text{RESOLVEPRIMARYCLASSFQN}(S_{cand}, f)$ 
22: if  $targetFqn$  is empty then
23:   $feedback \leftarrow$  "Test skipped: target class FQN could not be resolved."
24:   $useFeedbackOnly \leftarrow false$ 
25:  return failure
26: end if
27:  $TR \leftarrow \text{RUNANDSUMMARIZETESTS}(proj, targetFqn, S_{cand})$ 
28: if  $TR.status \neq tests\_passed$  then
29:   $feedback \leftarrow (TR.summary \text{ if exists else } TR.raw)$ 
30:   $useFeedbackOnly \leftarrow false$ 
31:  return failure
32: end if
33: if CLONEMETHODMODIFIED( $proj, f, watchedMethods$ ) then
34:  CANCELWORKFLOW
35:  return failure
36: end if
37: if USERCONFIRMSAPPLYDIFF( $S_{cur}, S_{cand}$ ) then
38:   $\text{WRITESOURCETOFILE}(f, S_{cand})$ 
39:  return success
40: else
41:  return success ▷ verified but not applied
42: end if

```

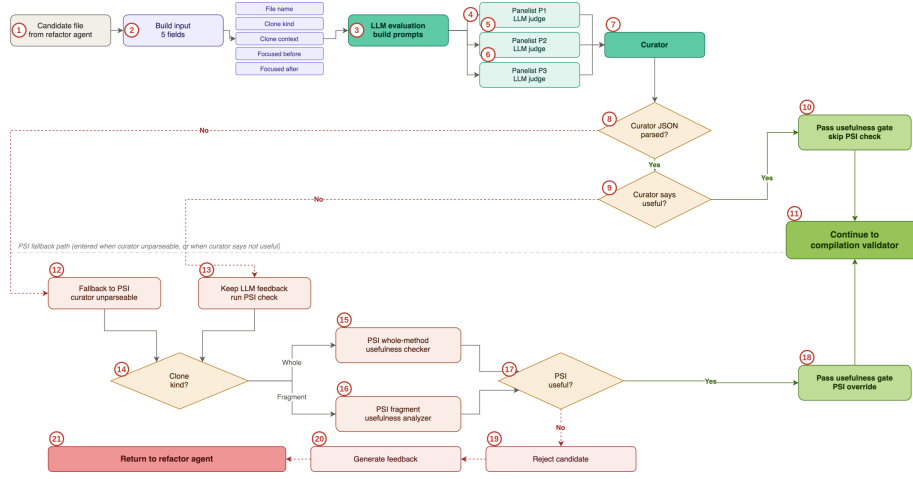


Figure 4: Usefulness Checker Pipeline.

received as input (1). The workflow then constructs the usefulness evaluation context using five fields: the file name, clone kind, clone context, focused before-refactoring code, and focused after-refactoring code (2). Based on this information, the system builds the prompts for the LLM-based usefulness evaluation stage (3). Three independent usefulness panelists then analyze the candidate refactoring separately and determine whether the transformation represents a useful Extract Method refactoring according to the predefined unuseful refactoring categories (4–6). The outputs from all panelists are then aggregated by the usefulness curator (7).

After collecting the panelist outputs, the workflow first checks whether the curator response can be parsed correctly as a valid structured JSON object (8). If the curator output is valid, the workflow then determines whether the curator considers the refactoring useful (9). Candidates that pass the LLM usefulness stage immediately pass the usefulness gate and skip the PSI fallback analysis (10), after which the workflow continues to the Compilation Validator (11). However, if the curator output is invalid or unparsable, the workflow falls back to the PSI-based usefulness analysis (12). Similarly, if the curator determines that the refactoring is not useful, the workflow preserves the LLM feedback and continues with PSI validation (13).

The PSI stage first determines whether the detected clone corresponds to a whole-method clone or a fragment clone (14). Whole-method clones are evaluated using the PSI whole-method usefulness checker (15), while fragment clones are evaluated using the PSI fragment usefulness analyzer (16). The workflow then determines whether the PSI analysis considers the refactoring useful (17). If the PSI analysis validates the refactoring, the candidate passes the usefulness gate through PSI override and proceeds to the Compilation Validator (18). Otherwise, the candidate refactoring is rejected (19), summarized feedback is

generated (20), and the feedback is returned to the Refactoring Agent for iterative repair in later attempts (21).

Listing 6: Usefulness Panelist prompt.

```
You are <PANELIST_TITLE> (<PANELIST_ID>).
Your task is to evaluate whether a proposed Java refactoring is useful.
You are responsible for the full category catalog listed below.
All panelists review the same categories independently.
If none of your assigned categories clearly apply, you must return
    is_useful=true.
Return ONLY one JSON object and no extra text.

The input contains the target clone context, the focused before-
    refactoring code, and the focused after-refactoring code.

Assigned not-useful categories:
- EXTRACT_METHOD_NOT_FOUND: No valid Extract Method outcome is visible in
    the refactored code for the target clone.
- INCOMPLETE_REFACTORING_DETECTED: Some duplicated logic still remains in
    the original target methods after refactoring.
- EXTRACTION_WITHOUT_CLONE_REPLACEMENT_DETECTED: A helper was added, but
    the original duplicated body still remains instead of being replaced
    .
- DIRECT_CLONE_REMOVAL_DETECTED: A clone occurrence was removed directly
    instead of extracting shared logic.
- POST_EXTRACTION_CLONE_DELETION_DETECTED: A helper was introduced, but
    one target clone method was deleted afterward.
- CALL_BASED_CLONE_SUBSTITUTION_DETECTED: One target clone now calls
    another existing method instead of using a new shared helper.
- CLONE_REMOVAL_BY_DELEGATION_DETECTED: The change relies on delegation
    without clearly extracting one shared helper for all target clones.
- FRAGMENTATION_OF_LOGIC_DETECTED: Shared logic was split across multiple
    helpers instead of one coherent extraction.
- NON_TARGET_CLONE_REFACTORING_DETECTED: The refactoring seems to target
    a different duplication while the intended target remains unresolved
    .
- EXCESSIVE_REFACTORING_DETECTED: The extracted helper pulls in code
    beyond the intended cloned fragment.

Input:
<USEFULNESS_EVALUATION_INPUT>

Output:
Return ONLY this JSON object:
{
    "panelist_id": "<PANELIST_ID>",
    "is_useful": true,
    "matched_categories": [],
    "summary": short decision summary,
```

```

    "feedback": specific revision instruction for the refactoring agent if
        not useful; otherwise empty string
}

Rules:
- Use ONLY the assigned categories in matched_categories.
- If you are not confident any assigned category applies, set is_useful=
    true and matched_categories=[].
- feedback should be actionable and concise.

```

Listing 6 presents the prompt used by the usefulness panelists in our LLM driven workflow. Each panelist independently evaluates whether the generated refactoring represents a useful Extract Method transformation based on a predefined catalog of known unuseful refactoring patterns identified in prior work [30]. The prompt provides both the focused before-refactoring and after-refactoring code context, allowing the panelists to reason about how the duplicated logic changed after refactoring. Instead of relying only on compilation or testing outcomes, the panelists explicitly analyze structural properties of the transformation, such as whether duplicated logic still remains, whether clone instances were improperly deleted, or whether the extracted helper introduces excessive unrelated logic. Each panelist returns a structured JSON response containing the usefulness decision, matched categories, summary, and actionable feedback for iterative repair. This design enables the workflow to combine multiple independent LLM judgments before the curator agent aggregates the final usefulness decision.

Listing 7: Usefulness Curator prompt.

```

You are the usefulness curator.

You must review three panelist outputs and make the final useful/not-
useful decision.

Apply a majority vote across the three panelists.
- If 2 or more panelists set is_useful=false with at least one matched
  category, set is_useful=false.
- If 2 or more panelists set is_useful=true, set is_useful=true.
- If there is no majority, use your own judgment based on the strength of
  the evidence. Prefer is_useful=true unless the flagged category is
  a critical structural issue such as INCOMPLETE_REFACTORING_DETECTED
  or EXTRACT_METHOD_NOT_FOUND.
Return ONLY one JSON object and no extra text.

If all parsed panelists say useful or only provide weak/empty evidence,
  set is_useful=true.

Return ONLY one JSON object and no extra text.

=== TARGET CONTEXT ===

```

```
File: <FILE_NAME>

Clone kind: <CLONE_KIND>

<CLONE_CONTEXT>

=== BEFORE (FOCUSED CONTEXT) ===

<FOCUSED_BEFORE_CODE>

=== AFTER (FOCUSED CONTEXT) ===

<FOCUSED_AFTER_CODE>

=== PANELIST OUTPUTS ===

[P1]

{
  "parsed": <P1_PARSED>,
  "is_useful": <P1_IS_USEFUL>,
  "matched_categories": <P1_MATCHED_CATEGORIES>,
  "summary": "<P1_SUMMARY>",
  "feedback": "<P1_FEEDBACK>",
  "error": "<P1_ERROR>"
}

[P2]

{
  "parsed": <P2_PARSED>,
  "is_useful": <P2_IS_USEFUL>,
  "matched_categories": <P2_MATCHED_CATEGORIES>,
  "summary": "<P2_SUMMARY>",
  "feedback": "<P2_FEEDBACK>",
  "error": "<P2_ERROR>"
}

[P3]

{
  "parsed": <P3_PARSED>,
  "is_useful": <P3_IS_USEFUL>,
  "matched_categories": <P3_MATCHED_CATEGORIES>,
  "summary": "<P3_SUMMARY>",
  "feedback": "<P3_FEEDBACK>",
  "error": "<P3_ERROR>"
}

=== OUTPUT JSON ===
```

```

{
  "is_useful": false,
  "reasons": ["CATEGORY_NAME"],
  "summary": short final summary,
  "feedback": merged revision guidance for the refactoring agent if not
    useful; otherwise empty string,
  "confidence": 0.84
}

Rules:

reasons must use only category names found by panelists.
Keep summary short and concrete.
feedback should directly tell the refactoring agent how to revise the
  code.

```

Listing 7 presents the curator prompt used to finalize the usefulness decision from multiple panelists. While each panelist independently analyzes the refactoring and reports matched unuseful categories, the curator is responsible for combining these judgments into a single structured decision used by the workflow. The curator reviews all panelist outputs, identifies overlapping or conflicting judgments, and produces the final usefulness label together with summarized feedback for the refactoring agent. This design reduces reliance on a single LLM judgment and improves decision stability by incorporating multiple independent evaluations before generating the final result. The curator also consolidates revision guidance from the panelists into concise actionable feedback, enabling the workflow to iteratively refine the refactoring in later attempts.

The PSI usefulness check uses two complementary signals: (1) normalized token similarity to determine whether clone-like code still remains after refactoring, and (2) structural evidence extracted from IntelliJ PSI to understand how the refactoring modifies the program structure. In particular, the checker examines whether the original target clone methods or fragments are preserved, whether a new helper method is introduced, whether target locations call the helper, and whether the transformation corresponds to a valid Extract Method rather than one of the known unuseful patterns.

We use two complementary techniques because each captures a different aspect of structural validity under the Extract Method definition used in this study. Normalized token similarity [31] measures whether duplicated logic still exists after refactoring. For example, if two methods remain highly similar after extraction (even after renaming variables or changing literals), it indicates that duplication was not effectively removed. However, token similarity alone cannot explain how the refactoring was performed. Therefore, we also use PSI-based structural analysis to inspect program structure. For instance, PSI allows us to detect whether both clone instances were replaced by calls to the same newly introduced helper method, which is expected in a proper Extract Method refactoring. By combining similarity measurement with structural evidence, we

can distinguish between true abstraction and superficial code changes.

Before any structural analysis, we use IntelliJ PSI. PSI is an abstract syntax representation that the IntelliJ platform uses for source code. PSI represents the source code as structured data. An element of PSI represents any concrete construct of the source code, such as a class, method, statement, or expression. PSI represents source code data with resolved information compared to raw text or token-based source code analysis. We use PSI for better analysis of refactorings at the source code level rather than at the surface level.

Step 0: Parse PSI and compute method-level differences. First, we parse the source file in both its before and after versions into IntelliJ PSI trees (lines 1–2 in Listing 8). This allows us to process the source file in a structured form, rather than a raw string.

Next, we gather all method declarations from the two PSI trees (lines 4–5). The methods are stored under unique keys, so we can easily compare the two versions.

Then, we find the set of newly added methods (lines 7–8). We do this by taking all methods from the later version and subtracting the methods that were already present in the earlier version.

This step is necessary because a proper Extract Method refactoring should introduce a new helper method. If no new method is detected at this stage, it is unlikely that a standard Extract Method has been applied. Listing 8 shows the implementation details.

Listing 8: PSI parsing and added method detection.

```
1 PsiJavaFile beforePsi = parseInMemoryJavaFile(project, fileName,
    beforeSource);
2 PsiJavaFile afterPsi = parseInMemoryJavaFile(project, fileName,
    afterSource);
3
4 Map<String, PsiMethod> beforeMethods = collectAllMethodsByKey(beforePsi);
5 Map<String, PsiMethod> afterMethods = collectAllMethodsByKey(afterPsi);
6
7 Set<String> addedKeys = new HashSet<>(afterMethods.keySet());
8 addedKeys.removeAll(beforeMethods.keySet()); // AFTER - BEFORE
```

Step 1: Measure clone similarity using normalized tokens. To determine whether duplication is removed, we adopt the Jaccard similarity because it quantifies the proportion of shared elements between two sets while ignoring duplicate occurrences. This makes it suitable for detecting structural similarity across normalized token sets [26].

During tokenization, all identifiers are normalized to ID (identifier), and all literals are normalized to LIT (literal). This normalization removes superficial differences such as variable names and constants, while preserving structural similarity.

Listing 9 shows how similarity is computed for both the before and after versions. In lines 1–3, we first build normalized signatures for the two clone instances in the previous version and compute their Jaccard similarity as `simBefore`. In lines 5–7, we repeat the same process for the after version and compute `simAfter`. By comparing `simBefore` and `simAfter`, we can determine whether the duplicated logic still remains after refactoring.

Listing 9: Token similarity with Jaccard.

```

1 Sig aBefore = buildSig(beforeMethods.get(aKey));
2 Sig bBefore = buildSig(beforeMethods.get(bKey));
3 double simBefore = jaccard(aBefore, bBefore);
4
5 Sig aAfter = buildSig(afterMethods.get(aKey));
6 Sig bAfter = buildSig(afterMethods.get(bKey));
7 double simAfter = jaccard(aAfter, bAfter);

```

Step 2: Extract PSI evidence about delegation and method calls. The token similarity alone cannot explain how the refactoring was performed. Therefore, in Step 2 we analyze structural changes using PSI. Listing 10 shows the implementation details with line numbers.

In lines 1–2, we call `detectDelegate` on the two after-version clone methods. This step checks whether a method becomes a thin delegate, meaning its body mainly forwards execution through a method call. A thin delegate is an important signal of Extract Method refactoring, because a proper extraction typically replaces duplicated logic with a call to a shared helper method.

In lines 5–6, we use `collectResolvedCalleeKeys` to collect the callee sets. Even though a method is not a pure delegate method, we still want to examine which methods are called within its body.

Listing 10: Delegation and callee collection.

```

1 DelegateInfo aDel = detectDelegate(aAfter, afterPsi, cfg);
2 DelegateInfo bDel = detectDelegate(bAfter, afterPsi, cfg);
3
4 // When not pure delegate, still collect callees in the body.
5 Set<String> aCalls = collectResolvedCalleeKeys(aAfter, afterPsi, cfg);
6 Set<String> bCalls = collectResolvedCalleeKeys(bAfter, afterPsi, cfg);

```

Through the combination of the detection of delegation (lines 1-2) and the collection of callees (lines 5-6), it is possible to infer whether the duplicated code has been replaced by a call to a common method, redirected to an existing method, or has been structurally altered in some other fashion. These PSI-based features enable us to differentiate between actual Extract Method refactoring and other superficial structural changes.

Using the structural information gathered in Step 2, the common failure modes identified in previous work [30] can be described.

Table 1: Definitions of LLM refactoring hallucination categories

Category	Definition	General Solution
<i>Incomplete refactoring</i>	The refactoring modifies the code, but most of the duplicated logic still remains in the original methods.	Ensure both clone instances are replaced by calls to the same newly extracted helper method, removing the duplicated logic entirely.
<i>Excessive refactoring</i>	The extracted method includes statements beyond the intended cloned fragment.	Restrict extraction strictly to the duplicated region and avoid introducing unrelated logic into the helper method.
<i>Post-extraction clone deletion</i>	A helper method is introduced, but only one clone is replaced by a call, while the other clone is deleted.	Replace both clone instances with calls to the extracted method instead of deleting one side.
<i>Direct clone removal</i>	One clone instance is deleted without introducing a shared abstraction.	Introduce a common helper method and redirect both clones to it instead of removing code directly.
<i>Call-based clone substitution</i>	One clone is replaced by a call to an existing method instead of extracting a new shared abstraction.	Introduce a dedicated extracted method that captures the shared logic, rather than redirecting to an existing clone.
<i>Fragmentation of logic</i>	The duplicated logic is split across several small helper methods instead of one coherent abstraction.	Extract the duplicated fragment into a single coherent helper method to centralize shared behavior.
<i>Clone removal by delegation</i>	The original method is modified to delegate behavior, and the clone calls this modified method instead of a new abstraction.	Introduce a separate extracted method that encapsulates only the duplicated fragment and let both methods delegate to it symmetrically.

Listing 11: Incomplete refactoring example

```

/* Code fragment 1 in LineAndShapeRenderer */
double previous = previousValue.doubleValue();
double x0;
if (this.useSeriesOffset) {
    x0 = domainAxis.getCategorySeriesMiddle(
        dataset.getColumnKey(column - 1),
        dataset.getRowKey(row), dataset,
        this.itemMargin, dataArea,
        plot.getDomainAxisEdge());
}
else {
    x0 = domainAxis.getCategoryMiddle(column - 1,

```



```

        getColumnCount(), dataArea,
        plot.getDomainAxisEdge());
    }
    double y0 = rangeAxis.valueToJava2D(previous, dataArea,
        plot.getRangeAxisEdge());
    Line2D line = null;
    if (orientation == PlotOrientation.HORIZONTAL) {
        line = new Line2D.Double(y0, x0, y1, x1);
    }
    else if (orientation == PlotOrientation.VERTICAL) {
        line = new Line2D.Double(x0, y0, x1, y1);
    }
    g2.setPaint(getItemPaint(row, column));
    g2.setStroke(getItemStroke(row, column));
    g2.draw(line);

    /* Code fragment 2 in StatisticalLineAndShapeRenderer */
    double previous = previousValue.doubleValue();
    double x0;
    if (getUseSeriesOffset()) {
        x0 = domainAxis.getCategorySeriesMiddle(
            dataset.getColumnKey(column - 1),
            dataset.getRowKey(row), dataset,
            getItemMargin(), dataArea,
            plot.getDomainAxisEdge());
    }
    else {
        x0 = domainAxis.getCategoryMiddle(column - 1,
            getColumnCount(), dataArea,
            plot.getDomainAxisEdge());
    }
    double y0 = rangeAxis.valueToJava2D(previous, dataArea,
        plot.getRangeAxisEdge());
    Line2D line = null;
    if (orientation == PlotOrientation.HORIZONTAL) {
        line = new Line2D.Double(y0, x0, y1, x1);
    }
    else if (orientation == PlotOrientation.VERTICAL) {
        line = new Line2D.Double(x0, y0, x1, y1);
    }
    g2.setPaint(getItemPaint(row, column));
    g2.setStroke(getItemStroke(row, column));
    g2.draw(line);

    /* Extracted method */
    protected void drawLineBetweenPoints(Graphics2D g2,
        PlotOrientation orientation,
        double x0, double y0,
        double x1, double y1,
        int row, int column) {

```

```

Line2D line = null;

if (orientation == PlotOrientation.HORIZONTAL) {
    line = new Line2D.Double(y0, x0, y1, x1);
}
else if (orientation == PlotOrientation.VERTICAL) {
    line = new Line2D.Double(x0, y0, x1, y1);
}

g2.setPaint(getItemPaint(row, column));
g2.setStroke(getItemStroke(row, column));
g2.draw(line);
}

```

Incomplete refactoring. As can be seen in Table 1, in an incomplete refactoring failure mode, an attempt is made to refactor the code, but the duplicated code is not effectively removed [30]. Listing 11 shows an example from a previous empirical study [30]. The extract method `drawLineBetweenPoints` does not fully extract the whole cloned code. In these instances, the output of the refactoring is structurally different but is still very similar to both instances of code duplication. Therefore, it is clear that the refactoring process did not actually accomplish its primary objective: removing code duplication using a common abstraction.

In terms of practicality, an incomplete refactoring is a failure mode because it does not actually offer a structural advantage. For instance, a model could add a new helper method, make method calls, but still leave both clone bodies almost entirely intact.

To detect incomplete refactoring, we compare the similarity of the clone instances before and after refactoring using normalized token-based Jaccard similarity. If the similarity in the after version remains high, this means the clone is still present.

In our implementation, we use a similarity threshold of 0.7 for the after-version normalized token Jaccard similarity (i.e., `simAfter` $\geq$ 0.7). We choose 0.7 because it means that at least 70% of the normalized tokens are shared between the two clone instances, which suggests that the duplicated logic remains largely unchanged. A lower threshold would make the check more sensitive, and it may label partially improved refactorings as incomplete. In contrast, a much higher threshold (e.g., 0.9 or above) would mostly detect near-identical fragments and could miss cases where duplication is still substantial. In general, 0.7 is a practical balance between sensitivity and strictness to identify incomplete refactorings.

However, high similarity alone is not sufficient, since no refactoring may have been attempted at all. Therefore, we additionally require evidence of a refactoring attempt derived from PSI analysis.

Specifically, we treat a refactoring attempt as having occurred if at least

one of the following conditions holds: (1) a new method is introduced in the after version, (2) at least one clone instance becomes a method call, or (3) the clone instances share one or more method calls that did not exist before. These signals indicate that the model tried to restructure the code, even if the result was ineffective.

Listing 12 shows the detection logic. In line 1, the algorithm first checks whether `simAfter` is greater than or equal to the predefined threshold. This condition indicates that the similarity between the two clone instances remains high after refactoring, suggesting that duplication may still exist.

In lines 2-5, the variable `refactorAttempt` is calculated. This boolean expression is a collection of PSI-based structural evidence. Line 3 checks if new methods were added (`addedKeys`). Line 4 checks if either of the clone instances is a delegate method. Line 5 checks if the two clone instances share new method calls (`sharedCalls`). The overall evidence points to a model refactoring attempt.

In lines 7-9, the two conditions are combined. If the similarity is still high (line 1) and there is structural evidence for a refactoring attempt (lines 2-5), then the refactoring is classified as `INCOMPLETE_REFACTORING`. The decision is returned in line 8, marked as not useful in a `PairOutcome`.

This design is conservative in that we are not penalizing cases where no refactoring attempt is made, and we are still effectively filtering out cases where there is a structural refactoring attempt that failed to meaningfully eliminate duplication.

Listing 12: Incomplete refactoring detection.

```

1  if (simAfter >= cfg.cloneSimilarityAfterStill) {
2      boolean refactorAttempt =
3          (!addedKeys.isEmpty()) ||
4          aDel.isDelegate || bDel.isDelegate ||
5          (!sharedCalls.isEmpty());
6
7      if (refactorAttempt) {
8          return new PairOutcome(Schema.INCOMPLETE_REFACTORING, false);
9      }
10 }

```

Excessive refactoring. Excessive refactoring refers to a situation where the extracted helper method is larger than the original cloned fragments. This kind of refactoring may compile and run successfully, but it is not a proper situation for an Extract Method refactoring.

To identify this kind of refactoring, we can start by obtaining the size of the original clone fragments (Lines 1-3, Listing 13). We can get this by counting the number of non-empty code lines of each clone fragment. Then we can get the larger one as a baseline.

Then we can get the extracted helper method by PSI (Lines 6-15). We can get this by intersecting the callees of two clone instances (Lines 6-7) and checking if any of these callees is a newly added method (Lines 10-14).

Finally, we compare the size of the extracted method body with the baseline (Lines 17–22). We allow a small tolerance of two lines (Line 21) to ignore trivial formatting differences. If the extracted method contains more lines than the baseline plus tolerance (Line 22), we classify the refactoring as *Excessive Refactoring* and mark it as not useful (Lines 23–30).

Listing 13 shows the complete detection logic.

Listing 13: Excessive refactoring detection.

```

1  int baseLinesA = countNonEmptyCodeLines(beforeA);
2  int baseLinesB = countNonEmptyCodeLines(beforeB);
3  int baseLines = Math.max(baseLinesA, baseLinesB);
4
5  // Identify extracted method: shared callee that is newly added
6  Set<String> shared = new HashSet<>(evA.calleeKeys);
7  shared.retainAll(evB.calleeKeys);
8
9  String extractedKey = null;
10 for (String k : shared) {
11     if (addedMethodKeys.contains(k)) {
12         extractedKey = k;
13         break;
14     }
15 }
16
17 if (extractedKey != null) {
18     PsiMethod extracted = afterMethods.get(extractedKey);
19     int extractedLines = countMethodBodyNonEmptyLines(extracted);
20
21     int tolerance = 2; // ignore trivial formatting differences
22     if (extractedLines > baseLines + tolerance) {
23         return new UsefulnessResult(
24             false,
25             35,
26             Strategy.EXCESSIVE_REFACTORING,
27             List.of(Reason.EXCESSIVE_REFACTORING_DETECTED),
28             "extractedLines=" + extractedLines +
29             ", baseFragmentLines=" + baseLines
30         );
31     }
32 }

```

Post-extraction clone deletion. Post-extraction clone deletion refers to a refactoring pattern where a helper method is introduced, but the duplicated code is not refactored symmetrically as shown in Table 1. Instead of replacing both clone instances with calls to the extracted method, only one clone instance is replaced, while the other clone instance is deleted. As a result, the duplicated logic is not uniformly abstracted into a single method.

Although a new helper method exists, the refactoring does not preserve the structures expected from Extract Method, where all duplicated instances should be replaced by calls to the same extracted method. Therefore, even though a new method is added, this refactoring is not considered useful under its definition.

To detect post-extraction clone deletion, we combine three signals from the after version, as shown in Listing 14. First, in Lines 1–2, we check whether each clone instance still exists by testing whether `aAfter` or `bAfter` is `null`. In Line 4, we use an exclusive-or condition to ensure that exactly one clone instance is deleted. This prevents cases where both sides are removed. Next, in Lines 5–6, we retrieve the remaining clone instance from `afterMethods`. In Lines 8–9, we invoke `detectDelegate` to determine whether the remaining method has become a delegate, meaning its body mainly consists of a method call. Finally, in Line 11, we check two additional conditions: whether new methods were introduced (`!addedKeys.isEmpty()`) and whether the remaining method is confirmed as a delegate. Only when all these conditions hold do we classify the refactoring as `POST_EXTRACTION_DELETION` in Line 12 and mark it as not useful.

Listing 14: Post-extraction clone deletion detection.

```

1 boolean aDeleted = (aAfter == null);
2 boolean bDeleted = (bAfter == null);
3
4 if (aDeleted ^ bDeleted) { // exactly one side deleted
5     String remainingKey = aDeleted ? bKey : aKey;
6     PsiMethod remaining = afterMethods.get(remainingKey);
7
8     DelegateInfo del = (remaining == null) ? null
9         : detectDelegate(remaining, afterPsi, cfg);
10
11     if (!addedKeys.isEmpty() && del != null && del.isDelegate) {
12         return new PairOutcome(Strategy.POST_EXTRACTION_DELETION, true);
13     }
14 }

```

Listing 15: Direct clone removal example

```

1 static public int direction(int x1, int y1, int x2, int y2) {
2     int direction = 0;
3     int vx = x2 - x1;
4     int vy = y2 - y1;
5     if (vy < vx && vx > -vy) {
6         direction = EAST;
7     }
8     else if (vy > vx && vy > -vx) {
9         direction = NORTH;
10    }
11    else if (vx < vy && vx < -vy) {
12        direction = WEST;

```

```

13     }
14     else {
15         direction = SOUTH;
16     }
17     return direction;
18 }
19 - static public int direction_Cloned(int x1, int y1,
20 - int x2, int y2) {
21 - int d = 0;
22 - int x = x2 - x1;
23 - int y = y2 - y1;
24 - if (y < x && x > -y) {
25 - d = EAST;
26 - }
27 - else if (y > x && y > -x) {
28 - d = NORTH;
29 - }
30 - else if (x < y && x < -y) {
31 - d = WEST;
32 - }
33 - else {
34 - d = SOUTH;
35 - }
36 - return d;
37 - }

```

Direct clone removal. In Table 1, direct clone removal refers to cases where duplicated code is eliminated by simply deleting one or both clone instances, without introducing any shared abstraction. Example from prior work [30] is shown in Listing 15. In other words, the refactoring removes code rather than restructuring it. This pattern was also reported as a common outcome that reduces duplication but does not follow the Extract Method pattern.

From a refactoring perspective, direct clone removal is different from Extract Method because no helper method is created to encapsulate the shared logic. As a result, there is no reusable abstraction that captures the intent of the duplicated code. Therefore, although duplication may be reduced, such changes are not considered useful in our workflow based on the definition of Extract Method refactoring.

To detect direct clone removal, we compare the presence of clone instances between the before- and after versions. If a clone instance cannot be located in the after version, we treat it as deleted. This can happen in two ways: either one side is removed, or both sides are completely removed.

However, deleting one side after introducing a helper method corresponds to a different pattern, namely post-extraction clone deletion. Therefore, we first check and exclude the post-extraction case. If the removal cannot be explained by post-extraction deletion, we classify it as direct clone removal.

Listing 16 shows the detection logic in detail.

In Line 1, the code checks whether exactly one clone instance is deleted using the exclusive-or condition. This means that one side is missing while the other still exists. If this condition holds, and the case was not already classified as post-extraction clone deletion, the refactoring is immediately labeled as `DIRECT_REMOVAL` in Line 3.

In Line 5, the second branch checks whether both clone instances are deleted using the condition. If both sides are removed, the refactoring is again classified as `DIRECT_REMOVAL` in Line 6. In both branches (Lines 3 and 6), the returned result marks the refactoring as not useful, because no shared helper method is introduced and no Extract Method structure is formed.

Listing 16: Direct clone removal detection.

```

1 if (aDeleted ^ bDeleted) {
2     // if not classified as POST_EXTRACTION_DELETION above
3     return new PairOutcome(Strategy.DIRECT_REMOVAL, true);
4 }
5 if (aDeleted && bDeleted) {
6     return new PairOutcome(Strategy.DIRECT_REMOVAL, true);
7 }

```

Call-based clone substitution. As listed in Table 1, call-based clone substitution refers to cases where both clone instances are replaced by calls to the same existing method, rather than to a newly extracted helper method. Listing 1 also shows an example of this pattern from previous work [30]. In this situation, the duplicated logic is removed from the two clone locations, but no new abstraction is introduced by the refactoring. Instead, the refactoring simply redirects both clone sites to an already-existing implementation.

Although duplication is reduced, it does not correspond to an Extract Method refactoring, because the refactoring does not create a new method that captures the shared logic. Therefore, we treat call-based clone substitution as not useful for our workflow.

To detect this case, we rely on PSI-based delegation and callee analysis. First, we check whether both clone sides become delegate methods after refactoring. A delegate method is a method whose body mainly consists of method calls rather than original logic. Second, we collect the set of resolved callee methods for each clone side and compute their intersection. If both sides dele-

gate to at least one common callee, this indicates that the duplicated logic has been substituted by a method call.

However, to distinguish this case from a proper Extract Method, we further check whether the shared callee corresponds to a newly added method. If all shared callees already existed in the previous version of the file, then the refactoring did not introduce a new abstraction. In this case, we classify the refactoring as call-based clone substitution.

Listing 17 shows the implementation in detail. In Line 1, the code first checks whether both clone sides are delegate methods using the condition. This ensures that both methods mainly consist of calls rather than original logic. In Lines 2–3, it computes the intersection of the callee sets from both sides. The variable `shared` contains the methods that are invoked by both clone instances. In Line 5, the condition `!shared.isEmpty()` verifies whether the two sides share at least one common callee. In Line 6, the code checks whether any of the shared callees correspond to newly added methods using `anyMatch(addedKeys::contains)`. Finally, in Lines 8–9, if none of the shared callees are newly introduced (i.e., `!sharedHasNew`), the refactoring is classified as `DELEGATION_TO_EXISTING`. In this case, the duplicated logic is redirected to an existing method rather than extracted into a new abstraction, and the refactoring is therefore marked as not useful.

Listing 17: Call-based clone substitution detection.

```

1 if (aDel.isDelegate && bDel.isDelegate) {
2     Set<String> shared = new HashSet<>(aDel.calleeKeys);
3     shared.retainAll(bDel.calleeKeys);
4
5     if (!shared.isEmpty()) {
6         boolean sharedHasNew = shared.stream().anyMatch(addedKeys::
7             contains);
8
9         if (!sharedHasNew) {
10             return new PairOutcome(Strategy.DELEGATION_TO_EXISTING, true)
11         }
12     }
13 }

```

Clone removal by delegation. Clone removal by delegation describes cases where delegation is applied in an asymmetric or inconsistent way in clone instances, as shown in Table 1. Instead of both clone locations being replaced by calls to the same extracted method, delegation happens only on one side, or the two sides delegate to different helper methods. As a result, duplication is not unified into a single abstraction, and the refactoring does not correspond to a proper Extract Method.

This pattern is part of the non-useful refactoring outcomes, and we explicitly detect it to avoid accepting such changes. The key observation is that delegation

alone is not sufficient: both clone sides must delegate to the same callee to truly remove duplication.

Our implementation relies on PSI-based call resolution. For each clone side in the after version, we collect the set of resolved callees appearing in the method body. We then compare the two callee sets to check whether there exists a shared callee that both sides invoke.

Listing 18 presents the detection logic with explicit line numbers. In Lines 1–2, we compute the intersection of the callee sets from both clone sides. The set `sharedCalls` contains the methods that are invoked by both instances in the after version. In Line 4, the boolean variable `anyDelegate` checks whether at least one side is classified as a delegate method (i.e., its body mainly consists of method calls). In Line 5, `noSharedCallee` verifies whether the two sides do not share any common callee. Finally, in Lines 7–8, if at least one side delegates (`anyDelegate`) and there is no shared callee between them (`noSharedCallee`), the refactoring is classified as clone removal by delegation and marked as not useful.

This condition captures asymmetric or inconsistent delegation, where delegation is applied but does not unify duplicated logic into a single shared abstraction.

Listing 18: Clone removal by delegation.

```

1 Set<String> sharedCalls = new HashSet<>(aCalls);
2 sharedCalls.retainAll(bCalls);
3
4 boolean anyDelegate = aDel.isDelegate || bDel.isDelegate;
5 boolean noSharedCallee = sharedCalls.isEmpty();
6
7 if (anyDelegate && noSharedCallee) {
8     return new PairOutcome(Strategy.UNKNOWN, false);
9 }

```

Fragmentation of logic. Table 1 shows the definition of fragmentation of logic. It refers to cases where a duplicated computation is no longer centralized into a single extracted method, but instead scattered into multiple helper calls. Rather than reducing duplication through one clear Extract Method refactoring, the logic becomes split across several smaller methods, making the refactoring harder to understand and maintain. This refactoring outcome was explicitly reported as an undesired pattern in prior work [30], and motivates us to detect and filter it.

In our workflow, we detect fragmentation by analyzing method call structure in the after version. Specifically, we examine which helper methods are called by the two clone locations after refactoring. If both sides invoke multiple shared callees, this indicates that the original duplicated logic has been decomposed into several pieces rather than extracted as a single cohesive unit.

Listing 19 presents the detection logic. In Lines 1–2, we compute the intersection of the resolved callee sets for the two clone sides (`aCalls` and `bCalls`).

The resulting set `sharedCalls` represents helper methods invoked by both sides after refactoring. In Lines 4–7, if the number of shared callees is at least two and the token-level similarity has been reduced, we classify the refactoring as fragmentation. In Lines 9–16, we additionally handle the case where both clone sides become delegate methods. If both are delegates and still share two or more callees (Lines 11–14), the transformation is again classified as fragmentation.

All cases detected through Lines 4–7 or Lines 11–14 are marked as not useful, since they do not correspond to a standard Extract Method refactoring that introduces a single shared abstraction.

Listing 19: Fragmentation of logic detection.

```

1 Set<String> sharedCalls = new HashSet<>(aCalls);
2 sharedCalls.retainAll(bCalls);
3
4 if (sharedCalls.size() >= 2 &&
5     simAfter <= cfg.cloneSimilarityAfterReduced) {
6     return new PairOutcome(Strategy.FRAGMENTATION, true);
7 }
8
9 if (aDel.isDelegate && bDel.isDelegate) {
10     Set<String> shared = new HashSet<>(aDel.calleeKeys);
11     shared.retainAll(bDel.calleeKeys);
12     if (shared.size() >= 2) {
13         return new PairOutcome(Strategy.FRAGMENTATION, true);
14     }
15 }

```

Final decision.. After classification, we apply a strict decision rule. Only refactorings that correspond to a standard Extract Method and do not fall into any of the above categories are considered useful. All other outcomes are rejected, even if the refactored code compiles and passes tests.

4.7. Retrieval-Augmented Generation

The workflow uses retrieval-augmented generation to support both clone detection and refactoring. The purpose of this component is to provide concrete examples from prior cases, rather than to define refactoring rules or make decisions. Retrieved examples are used as contextual guidance and are injected into prompts when available.

To support retrieval-augmented detection, we construct a clone example database used as few-shot context for the Detection Agent. The database is designed to expose the model to realistic clone patterns observed in prior tools and benchmarks, rather than to define explicit detection rules. For Type-1, Type-2, and Type-3 clones, we collect all the examples from JDeodorant [38] in nine open-source Java projects: Apache Columba, EMF, Hibernate, jEdit, JMeter, JRuby, Squirrel SQL Client, and JFreeChart. These projects are commonly used in clone research and contain developer-confirmed refactoring

cases. We further include Type-1 to Type-3 clone pairs from BigCloneBench [36] and GPT-Benchmark [2] to increase diversity and difficulty. To cover semantic clones (Type-4), we extend the database with examples from LLM-for-CCD [20] and BigCloneBench [36]. In GPT-Benchmark, clone types are classified using Claude Opus 4.5, since it was the newest model available at the time and showed better performance than GPT-5.1 on clone type classification. By adding those examples, the Detection Agent is exposed to a broad range of clone patterns while maintaining a consistent prompt structure.

We also construct a separate example database to support retrieval-augmented refactoring. This database is used only by the Refactoring Agent and contains concrete Extract Method refactoring examples. For Type-1, Type-2, and Type-3 clones, we randomly sample 50 clone instances for each type from the JDeodorant projects. Each selected clone instance includes the original duplicated code and its corresponding refactored version, where duplication is removed by extracting a shared helper method. For Type-4 (semantic) clones, we select 50 clone pairs from LLM-for-CCD. Because LLM-for-CCD does not provide refactoring ground truth, we apply Extract Method refactoring using GPT-5.1, which is different from the model used in our experiments because it was the newest model available when we created the RAG database. The generated refactorings are then manually inspected to verify that the extracted method is behavior-preserving and that duplication is correctly removed. Only refactorings that follow a standard Extract Method pattern and introduce a single shared abstraction are included. This process ensures that the refactoring database contains consistent, non-trivial examples across clone types.

Two separate example collections are used. For clone detection, the example collection consists of labeled pairs of code fragments. These pairs cover a wide range of cases, including Type-1, Type-2, Type-3, and Type-4 clones, as well as non-clone examples. Including both positive and negative cases allows the Detection Agent to observe different forms of similarity and to better distinguish true clones from coincidental resemblance.

For refactoring, a different example collection is used. Each entry contains a pair of cloned code fragments together with a refactored version produced using Extract Method. All examples in this collection follow the same refactoring strategy, which is consistent with the constraint enforced in the Refactoring Agent. These examples represent concrete refactoring results observed in practice.

The way retrieved examples are selected differs between detection and refactoring. In the detection stage, a small and fixed number of examples is included as few-shot demonstrations. The examples are chosen to cover different clone types as well as non-clone cases, and are inserted near the beginning of the detection prompt. This design keeps the prompt concise while still exposing the model to diverse detection scenarios.

In the refactoring stage, inspired by Mantra [40], the selection of examples is performed in two steps to strike a balance between efficiency and structural relevance.

In the first step, a simple lexical similarity test is performed by comparing

the number of common tokens in the query clone and the example code. The tokens are identified by normalizing the identifiers and literals in the code.

In the second step, a semantic similarity test is performed by converting the code into vector embeddings to calculate the cosine similarity between the query clone and the example code.

The results of both the tests are fused together using the reciprocal rank fusion (RRF) technique [13] to perform the final selection of the examples to be included in the prompt. Instead of using the similarity scores of the examples, the RRF technique relies on the rank of the examples in the final selection of k examples to be included in the prompt.

In the refactoring prompt, each selected example is presented in a consistent before-and-after format. The examples are clearly separated from the target source code and are explicitly marked as guidance only. The Refactoring Agent is instructed not to copy code directly from the examples, but to follow the demonstrated refactoring pattern.

Across both stages, the use of retrieved examples is optional. If no suitable examples are found, the workflow continues without retrieval-based generation. Correctness is not enforced by the retrieved examples, but by grounding checks in detection and by compilation and testing in refactoring.

4.8. Qualitative Analysis

This work is designed as a case study based on the ArgoUML project to better understand the behavior of LLM-based clone refactoring workflows under real-world clone scenarios. All experiments were performed manually by two authors because the evaluation required detailed inspection of generated refactorings, including usefulness validation, compilation outcomes, behavioral testing, and comparison against Extract Method criteria. For each clone case and each model configuration, the authors independently ran the models, applied the generated refactoring to the ArgoUML project, and recorded the outcomes, including detection, usefulness, compilation, testing, and final success.

We evaluated four language models: GPT-5.2, Gemini 3 Pro, Claude Opus 4.6, and Deepseek v3.2. These models were selected because they represent different model families and capability levels. By comparing these models under the same workflow, we are able to observe how model characteristics interact with structural constraints such as the usefulness checker and the feedback loop.

The usefulness evaluation followed the structural criteria defined in Section 4.6. Each refactoring was examined to determine whether it satisfied the Extract Method definition adopted in this study.

We also performed manual analysis on the cases where JDeodorant reported either *No Refactoring Opportunities* or *Precondition Violations*. The analysis was conducted independently by two authors to better understand how the LLM-generated refactorings differed from the decisions produced by JDeodorant. To better understand the cases where JDeodorant did not perform Extract Method refactoring, we manually validated all cases classified as either

No Refactoring Opportunities or *Precondition Violation*. The validation process was conducted independently by two authors to reduce subjective bias and improve the reliability of the analysis.

There are 10 cases categorized as *No Refactoring Opportunities*, we examined the outputs generated by all four LLM-based workflows from the earlier experiments, including the detection, usefulness, compilation, and testing results associated with each clone instance. If at least one LLM-generated refactoring successfully passed usefulness checking, compilation, and behavioral testing, we considered the clone to be practically refactorable under our evaluation criteria. This analysis helped us better understand the differences between the conservative refactoring decisions made by JDeodorant and the more flexible transformations generated by LLM-based workflows.

There are 41 cases categorized as *Precondition Violation*, the two authors also performed independent manual inspection. We first analyzed whether any LLM-based workflow successfully generated valid refactorings for the same clone instance based on detection, usefulness, compilation, and testing results. If successful cases existed, we further examined the corresponding workflow log files in detail. Specifically, we extracted the final refactored code generated by the LLM-based workflow and compared it against both the original methods and the precondition violations reported by JDeodorant. We then manually analyzed how the generated refactorings modified the extraction boundaries, parameter passing, statement organization, or variable dependencies to satisfy compilation and behavioral correctness while avoiding the structural conditions that triggered JDeodorant’s precondition violations. The identified handling strategies were subsequently recorded during the validation process.

However, if none of the LLM-based workflows successfully produced a useful, compilable, and behavior-preserving refactoring for a clone instance, the case was manually marked as not refactorable.

After the independent inspection phase, the two authors compared their evaluations and discussed disagreement cases until consensus was reached. The inter-rater agreement achieved a Cohen’s Kappa score of 1.0, indicating complete agreement on the final evaluation results.

5. Experimental Results

In this section, we discuss the results and findings from the experiments.

5.1. RQ1: How effective is the proposed workflow for clone detection and Extract Method refactoring in terms of correctness and usefulness when applied to real-world code clones?

5.1.1. Approach.

We evaluate the proposed workflow on real-world code clones extracted from the ArgoUML project [32]. To avoid potential data leakage, we exclude projects

Table 2: Multi-Agent Workflow Results on Real-World Clones (RQ1)

Model	Detected	PSI Detected	Useful	Compilation	Test	Success
Claude Opus 4.6	117 (97.5%)	2	112 (93.3%)	101 (84.2%)	88 (73.3%)	88 (73.3%)
DeepSeek v3.2	117 (97.5%)	0	109 (90.8%)	96 (80.0%)	84 (70.0%)	84 (70.0%)
GPT-5.2	97 (80.8%)	16	99 (82.5%)	85 (70.8%)	70 (58.3%)	70 (58.3%)
Gemini 3 Pro	116 (96.7%)	0	112 (93.3%)	102 (85.0%)	90 (75.0%)	90 (75.0%)
Overall	447 (93.1%)	18	432 (90.0%)	384 (80.0%)	332 (69.2%)	332 (69.2%)

Note. Detection is calculated over 120 real-world clone cases per model. Compilation, Test, and Success are computed over all evaluated cases. Percentages are calculated over 120 cases per model and 480 cases overall.

already included in the JDeodorant benchmark dataset because they are also used in our RAG examples. ArgoUML has been extensively used as a subject system in empirical studies on clone evolution and maintenance [9, 37, 7] as well as on feature-oriented refactoring and software product line extraction [14] To obtain candidate clones. We run Deckard on the project source code to automatically detect clone pairs.

To ensure consistency with our workflow design, we focus on the same-file whole method clones. Since Deckard may produce false positives, we manually inspect the randomly selected results and retain only true clone instances for evaluation. Each selected clone instance is then used as input to the LLM driven workflow for Extract Method refactoring.

To analyze the impact of clone size on refactoring performance, we divide the 120 selected clone instances into three categories based on the number of duplicated lines: small clones (1–5 lines), medium clones (6–10 lines), and large clones (more than 10 lines). Each clone group contains 40 clone instances. We intentionally select a variety of clone types, including Type-1, Type-2, and Type-3 clones, to ensure diversity in structural similarity and modification patterns. Because Deckard does not explicitly report clone types, the classification is determined manually based on the structural characteristics of each clone pair. Each selected clone instance is then used as input to the LLM driven workflow for Extract Method refactoring.

We evaluate the workflow using the metrics including usefulness of refactoring, compilation success, and test pass rates.

Detection. From the results presented in Table 2, the average percentage of detections in real-world clones stands at 93.1% (447/480), suggesting that the workflow is still highly reliable for detecting duplicate code. Nevertheless, when compared to the controlled environment of RQ1 where all the clones were successfully detected, there are still some duplicates that go undetected in the LLM-based detection approach.

The performance of the models demonstrates that Claude Opus 4.6 and DeepSeek v3.2 both achieve detection rates of 97.5%, while Gemini 3 Pro achieves 96.7%. On the contrary, GPT-5.2 shows relatively poor detection per-

Table 3: Performance on Small Real-World Clones (1–5 Lines)

Model	Detected	PSI Detected	Useful	Compilation	Test	Success
Claude Opus 4.6	39 (97.5%)	1	37 (92.5%)	34 (85.0%)	29 (72.5%)	29 (72.5%)
DeepSeek v3.2	39 (97.5%)	0	37 (92.5%)	33 (82.5%)	30 (75.0%)	30 (75.0%)
GPT-5.2	36 (90.0%)	3	32 (80.0%)	31 (77.5%)	26 (65.0%)	26 (65.0%)
Gemini 3 Pro	40 (100.0%)	0	38 (95.0%)	35 (87.5%)	32 (80.0%)	32 (80.0%)
Overall	154 (96.3%)	4	144 (90.0%)	133 (83.1%)	117 (73.1%)	117 (73.1%)

Note. Each model is evaluated on 40 small clone cases. Percentages are calculated over 40 cases per model and 160 cases overall. The number of useful refactorings may exceed the number of detected clones. This is because clones not detected by the LLM-based detector are subsequently identified by the PSI-based fallback detector, and are still processed by the refactoring workflow.

Table 4: Performance on Medium Real-World Clones (6–10 Lines)

Model	Detected	PSI Detected	Useful	Compilation	Test	Success
Claude Opus 4.6	39 (97.5%)	1	38 (95.0%)	36 (90.0%)	33 (82.5%)	33 (82.5%)
DeepSeek v3.2	39 (97.5%)	0	37 (92.5%)	32 (80.0%)	27 (67.5%)	27 (67.5%)
GPT-5.2	32 (80.0%)	4	33 (82.5%)	28 (70.0%)	25 (62.5%)	25 (62.5%)
Gemini 3 Pro	37 (92.5%)	0	37 (92.5%)	34 (85.0%)	32 (80.0%)	32 (80.0%)
Overall	147 (91.9%)	5	145 (90.6%)	130 (81.3%)	117 (73.1%)	117 (73.1%)

Note. Each model is evaluated on 40 medium clone cases. Percentages are calculated over 40 cases per model and 160 cases overall. The number of useful refactorings may exceed the number of detected clones. This is because clones not detected by the LLM-based detector are subsequently identified by the PSI-based fallback detector, and are still processed by the refactoring workflow.

formance with only 80.8%.

In examining detection rates based on varying clone sizes, the following trends emerge. As shown in Table 3, small clones, consisting of 1–5 lines, have a detection rate of 96.3% (154/160). This suggests that even very small duplicated fragments can be detected effectively. Medium clones, ranging from 6–10 lines, yield a detection rate of 91.9% (147/160) from Table 4.

In contrast, detection performance drops for large clones (10+ lines), where the overall rate decreases to 91.3% (146/160) shown in Table 5. This decline is primarily driven by GPT-5.2, which detects only 72.5% of large clones, while other models remain relatively stable. The results suggest that longer clones introduce more complex control flow and contextual dependencies, making detection more challenging for some models.

Despite these missed detections, the overall workflow is not significantly affected because clones not identified by the LLM-based detector can still be recovered by the PSI-based fallback detector. This complementary design ensures that most clone instances are eventually processed in the refactoring stage,

Table 5: Performance on Large Real-World Clones (10+ Lines)

Model	Detected	PSI Detected	Useful	Compilation	Test	Success
Claude Opus 4.6	39 (97.5%)	0	37 (92.5%)	31 (77.5%)	26 (65.0%)	26 (65.0%)
DeepSeek v3.2	39 (97.5%)	0	35 (87.5%)	31 (77.5%)	27 (67.5%)	27 (67.5%)
GPT-5.2	29 (72.5%)	9	32 (80.0%)	26 (65.0%)	19 (47.5%)	19 (47.5%)
Gemini 3 Pro	39 (97.5%)	0	37 (92.5%)	33 (82.5%)	26 (65.0%)	26 (65.0%)
Overall	146 (91.3%)	9	141 (88.1%)	121 (75.6%)	98 (61.3%)	98 (61.3%)

Note. Each model is evaluated on 40 large clone cases. Percentages are calculated over 40 cases per model and 160 cases overall. The number of useful refactorings may exceed the number of detected clones. This is because clones not detected by the LLM-based detector are subsequently identified by the PSI-based fallback detector, and are still processed by the refactoring workflow.

reducing the impact of initial detection failures.

Listing 20: Example of Non-Clone Pair Detected by Deckard

```

public void actionPerformed(ActionEvent ae) {
    if (textSource == null) {
        if (removeFromDiagramAllowed()) {
            CutAction cmd =
                new CutAction(Translator.localize("action.cut"));
            cmd.actionPerformed(ae);
        }
    } else {
        textSource.cut();
    }
    if (isSystemClipboardEmpty()
        && Globals.clipBoard == null
        || Globals.clipBoard.isEmpty()) {
        ActionPaste.getInstance().setEnabled(false);
    } else {
        ActionPaste.getInstance().setEnabled(true);
    }
}

public void caretUpdate(CaretEvent e) {
    if (e.getMark() != e.getDot()) { // there is a selection
        setEnabled(true);
        textSource = (JTextComponent) e.getSource();
    } else {
        Collection figSelection =
            Globals.curEditor().getSelectionManager().selections();
        if (figSelection == null || figSelection.isEmpty()) {
            setEnabled(false);
        } else {
            setEnabled(true);
        }
    }
}

```



```

    }
    textSource = null;
  }
}

```

Listing 20 shows an example of a clone pair reported by Deckard that was manually discarded during our evaluation. Although Deckard identified the two methods as clones due to partial structural similarity and overlapping control-flow patterns, all LLM-based detectors consistently failed to recognize this pair as a meaningful clone instance. After manual inspection, we also determined that the duplicated content was limited. The two methods serve different functional purposes and contain substantially different logic despite sharing several syntactic patterns, such as conditional branches and enable/disable operations. This example illustrates what we mean by potential false positives in traditional clone detection tools.

Overall, the results demonstrate that LLM-based detection remains robust in real-world settings, while clone size and structural complexity introduce variability in detection performance across models.

Usefulness. As shown in Table 2, the overall usefulness rate on real-world clones is 90.0% (432/480), indicating that the workflow is effective in generating structurally valid Extract Method refactorings in realistic settings. Compared to detection, usefulness shows a noticeable drop, suggesting that identifying clones is easier than producing correct and structurally valid refactorings.

Across models, Claude Opus 4.6 and Gemini 3 Pro achieve the highest usefulness rate of 93.3%, followed by DeepSeek v3.2 at 90.8%. In contrast, GPT-5.2 shows a lower usefulness rate of 82.5%, indicating that it struggles more with generating structurally correct refactorings even when clones are detected.

When analyzing usefulness across different clone sizes, we observe clear variation. For small clones (1–5 lines), the overall usefulness rate is 90.0% (144/160), indicating that short duplicated fragments are not always straightforward to refactor due to ambiguous extraction boundaries. For medium-sized clones (6–10 lines), usefulness increases slightly to 90.6% (145/160), representing the highest performance among all groups. This suggests that medium clones provide clearer structural signals for identifying appropriate extraction boundaries.

In contrast, usefulness decreases for large clones (10+ lines), where the overall rate drops to 88.1% (141/160) compared to the medium group. Larger clones often involve more complex control flow and interleaved logic, making it more difficult to determine which parts should be extracted into a shared method and which parts should remain in the original methods.

Overall, the findings highlight that structural correctness remains a key challenge in real-world refactoring, and effectiveness depends not only on detecting duplication but also on correctly capturing the extraction structure.

Compilation. As shown in Table 2, the overall compilation success rate on real-world clones is 80.0% (384/480), indicating that most generated refactorings are syntactically valid and can be successfully compiled. However, compared

to usefulness (90.0%), compilation shows a further decrease, suggesting that even structurally valid refactorings may still contain syntax errors or unresolved dependencies in real-world code.

Across models, Gemini 3 Pro achieves the highest compilation rate of 85.0%, followed by Claude Opus 4.6 at 84.2% and DeepSeek v3.2 at 80.0%. GPT-5.2 again shows the lowest performance at 70.8%, indicating weaker robustness in producing compilable refactoring code.

When examining compilation performance across clone sizes, similar trends to usefulness are observed. For small clones (1–5 lines), the overall compilation rate is 83.1% (133/160), reflecting that short fragments may lead to incomplete method extraction or missing context required for compilation. For medium-sized clones (6–10 lines), the compilation rate decreases slightly to 81.3% (130/160), representing relatively stable performance across models. This suggests that medium clones generally provide sufficient context for generating syntactically complete refactorings.

In contrast, compilation performance drops further for large clones (10+ lines), where the overall rate decreases to 75.6% (121/160). Larger clones often involve more complex control flow and dependencies, increasing the likelihood of introducing compilation issues such as incorrect variable scope, missing parameters, or inconsistent method signatures.

Overall, the results indicate that while many refactorings pass structural checks, ensuring that all dependencies and method contexts are correctly handled remains a challenge in real-world scenarios.

Testing. As shown in Table 2, the overall test pass rate on real-world clones is 69.2% (332/480), representing the final stage of validation in the workflow. Compared to compilation (80.0%), the test pass rate further decreases, indicating that even compilable refactorings may still introduce behavioral inconsistencies.

Across models, Gemini 3 Pro achieves the highest test pass rate of 75.0%, followed by Claude Opus 4.6 at 73.3% and DeepSeek v3.2 at 70.0%. GPT-5.2 again shows the lowest performance at 58.3%, suggesting that it is more likely to produce refactorings that fail to preserve behavior under test execution.

For 41 clone cases where the original project did not provide sufficient automated test coverage, we additionally generated supplementary tests using GPT-5.5 to further validate the refactored code. The generated tests were manually reviewed before execution, and all evaluated refactorings successfully passed the generated test cases. This additional validation step was used only for cases with insufficient existing coverage and was intended to reduce the risk of overlooking behavioral inconsistencies that may not be exposed by the original test suite.

When examining performance across clone sizes, similar trends are observed. For small clones (1–5 lines), the overall test pass rate is 73.1% (117/160), indicating that even short refactorings may introduce subtle behavioral issues due to incorrect extraction boundaries. For medium-sized clones (6–10 lines), the test pass rate remains stable at 73.1% (117/160), representing the best overall balance between structural correctness and behavioral preservation. This suggests that medium clones provide sufficient context for both structurally correct

and behavior-preserving refactorings.

In contrast, the test pass rate drops for large clones (10+ lines), where the overall rate decreases to 61.3% (98/160). Larger clones often involve more complex interactions and dependencies, increasing the risk of introducing behavioral inconsistencies during refactoring, even when the code compiles successfully.

Overall, the results highlight that passing compilation does not guarantee behavioral correctness. While many refactorings are syntactically valid, preserving the original program behavior remains challenging, particularly for complex real-world clones.

Comparison Across Clone Sizes. To better understand the impact of clone size on workflow performance, we compare the results across small (1–5 lines), medium (6–10 lines), and large (10+ lines) clones using Table 3, Table 4, and Table 5. A consistent pattern emerges across all evaluation metrics.

As shown in Table 4, medium-sized clones achieve the best overall performance, with the highest usefulness (90.6%), compilation (81.3%), and test pass rates (73.1%). This indicates that medium clones provide sufficient structural context for both accurate detection and stable refactoring. The boundaries of duplicated code are clearer in this range, enabling the workflow to generate correct and complete Extract Method transformations.

Listing 21: Example of Medium-Sized Real-World Clone Pair

```
public Collection getChildren(Object parent) {
    if (parent instanceof AssociationsNode) {
        return Model.getCoreHelper()
            .getAssociations(((AssociationsNode) parent).getParent());
    }
    return Collections.EMPTY_SET;
}

public Set getDependencies(Object parent) {
    if (parent instanceof AssociationsNode) {
        Set set = new HashSet();
        set.add(((AssociationsNode) parent).getParent());
        return set;
    }
    return Collections.EMPTY_SET;
}
```

Listing 21 presents an example of a medium-sized real-world clone pair selected from the Deckard detection results. Unlike the manually constructed clone cases used in earlier experiments, this example reflects naturally occurring duplication patterns in production code. The two methods share similar control flow, conditional checks, and object access patterns, while still containing semantic differences in the returned data structures and operations. This level of similarity provides sufficient contextual information for the workflow to accurately identify the duplicated logic and generate stable Extract Method

refactorings. The example also illustrates why medium-sized clones generally produce stable refactoring outcomes across usefulness, compilation, and testing metrics. Compared to very small clones, the duplicated structure is clearer and less ambiguous, while compared to large clones, the code complexity and dependency interactions remain relatively manageable.

In contrast, Table 3 shows that small clones exhibit similar performance, with usefulness at 90.0% and test pass rate at 73.1%. Although small clones are easy to detect, they often lack sufficient context for reliable refactoring. The limited number of statements makes it harder to determine appropriate extraction boundaries, which can lead to incorrect refactorings.

Listing 22: Example of Small-Sized Real-World Clone Pair

```
public ActionState createActionState() {
    ActionState myActionState = modelImpl.getUmlPackage()
        .getActivityGraphs().getActionState().createActionState();
    super.initialize(myActionState);
    return myActionState;
}

public ActivityGraph createActivityGraph() {
    ActivityGraph myActivityGraph = modelImpl.getUmlPackage().
        getActivityGraphs().getActivityGraph().createActivityGraph();
    super.initialize(myActivityGraph);
    return myActivityGraph;
}
```

The following Listing 22 demonstrates a real-world instance of a small-sized clone pair discovered by Deckard. There is a strong similarity between the two procedures in terms of their structure, such as object instantiation, object initialization, and return value generation. Yet, since the duplication in question is rather short, the boundaries of its extraction become less clear-cut than those in the case of medium-sized clones. Small-sized clones typically contain little context information that would enable the pipeline to choose the statements eligible for moving into a helper function. Consequently, although such clones are straightforward to identify, they tend to result in either incomplete or structurally flawed Extract Method refactorings.

As shown in Table 5, large clones exhibit the lowest performance across most metrics, particularly in testing, where the pass rate drops to 61.3%. While larger clones provide more context, they also introduce increased complexity, including nested control flow, interleaved logic, and additional dependencies. These factors make it more difficult to identify precise extraction boundaries and to preserve behavior during refactoring.

Listing 23: Example of Large-Sized Real-World Clone Pair

```
private ActionList getDependencyToActions(
    final Object element, ArgoDiagram diagram) {
    ActionList al= new ActionList("Add_Dependencies_to");
```

```

Collection dependenciesTo =
    Model.getFacade().getClientDependencies(element);
for (Object dependency : dependenciesTo) {

    // Only show actions for associations not already on diagram
    if (diagram.presentationFor(dependency) == null) {
        Collection suppliers =
            Model.getFacade().getSuppliers(dependency);
        for (Object supplier : suppliers) {
            al.add(new AddUsageAction(
                "Add_dependency_to_"
                + Model.getFacade().getName(supplier),
                diagram, element, dependency, supplier));
        }
    }
}
return al;
}

private ActionList getDependencyFromActions(
    final Object element, ArgoDiagram diagram) {
    ActionList al= new ActionList("Add_dependencies_from_");

    Collection dependenciesFrom =
        Model.getFacade().getSupplierDependencies(element);
    for (Object dependency : dependenciesFrom) {

        // Only show actions for associations not already on diagram
        if (diagram.presentationFor(dependency) == null) {
            Collection clients =
                Model.getFacade().getClients(dependency);
            for (Object client : clients) {
                al.add(new AddUsageAction(
                    "Add_dependency_from_"
                    + Model.getFacade().getName(client),
                    diagram, element, dependency, client));
            }
        }
    }
    return al;
}

```

Example 23 shows a sample large clone found using the Deckard tool. Both methods have many similar logical constructs in them, such as loops inside loops, dependency tree traversal, conditionals, and multiple object creations. While there is no question about the duplication in the code, large clones present many more contextual complexities than small and medium clones. For instance, the refactoring process needs to preserve multiple interrelated statements, nested

Table 6: Precondition Violations Reported by JDeodorant on ArgoUML

Category	Description / Example	Count
Return/control-flow inconsistency	Unmatched return statements, conditional returns, or execution flows that do not terminate consistently.	13
Type unification failure	Expressions cannot be unified because the common superclass does not declare required members.	10
Dependency constraint violation	Expressions cannot be parameterized due to dependency constraints.	6
Control dependency violation	Statements cannot be moved because of control dependencies.	5
No refactoring opportunity found	No suitable Extract Method opportunity identified.	10
Void method parameterization issue	Void method calls cannot be parameterized.	4
Incompatible return types	Cloned fragments return incompatible variable types.	3
No matched statements	The number of matched statements is zero.	1
Total Unrefactorable Cases		51
Successful Refactorings		69

control flows, and object dependencies when extracting the code, leading to increased chances of breaking the original behavior of the code. This example clarifies how large clones suffer from inferior compilation and testing results, despite their relatively easier structural detection process. More specifically, the higher refactoring scale makes them highly vulnerable to the issues related to dependency resolution and variable scoping, among others.

Overall, the comparison across Table 3, Table 4, and Table 5 suggests that clone size influences refactoring effectiveness. Small and medium-sized clones show relatively similar performance across most metrics, while large clones consistently exhibit lower compilation and test success rates due to increased structural complexity and dependency interactions.

5.1.2. JDeodorant

To further understand the structural limitations of traditional Extract Method refactoring tools and compare them with LLM-based workflows, we additionally analyze the behavior of JDeodorant on the same real-world clone cases.

JDeodorant encounters precondition violations in 41 cases (34.2%), meaning that the tool refuses to apply Extract Method refactoring because the selected code fragments do not satisfy its internal structural constraints. In addition, there are 10 cases (8.3%) where no refactoring opportunities are identified.

Table 7: LLM Workflow Performance on Cases Successfully Refactored by JDeodorant

Model	Detected	Useful	Compilation	Test
JDeodorant	69 (100.0%)	69 (100.0%)	69 (100.0%)	69 (100.0%)
Claude Opus 4.6	69 (100.0%)	63 (91.3%)	56 (81.2%)	45 (65.2%)
DeepSeek v3.2	68 (98.6%)	65 (94.2%)	55 (79.7%)	49 (71.0%)
Gemini 3 Pro	69 (100.0%)	67 (97.1%)	58 (84.1%)	51 (73.9%)
GPT-5.2	68 (98.6%)	63 (91.3%)	58 (84.1%)	44 (63.8%)

Notes: Results are computed on the 69 clone cases where JDeodorant successfully applied Extract Method refactoring. Unlike the LLM-based workflows, JDeodorant only performs refactorings when all internal structural preconditions are satisfied, resulting in perfect compilation, testing, and usefulness rates on these cases.

Table 6 summarizes the major categories of precondition violations encountered by JDeodorant when applied to the 120 real-world clone cases from ArgoUML. The most common failure category is related to return and control-flow inconsistencies, accounting for 13 cases, followed by type unification failures with 10 cases. These results suggest that many clone instances contain structural dependencies, incompatible execution paths, or type constraints that prevent JDeodorant from safely applying Extract Method refactoring under its conservative rule-based analysis. Other common violations include dependency constraints, control dependencies, and parameterization limitations for void method calls. Overall, the table highlights the strict structural assumptions enforced by traditional refactoring tools and helps explain why many practically refactorable clone cases are rejected before refactoring is attempted.

However, for the remaining cases where JDeodorant successfully performs refactoring, the generated transformations achieve 100% compilation, testing, and usefulness rates. This indicates that JDeodorant is highly reliable once all structural preconditions are satisfied. Unlike LLM-based approaches, the tool applies conservative program analysis and only performs refactorings when correctness can be strongly guaranteed.

Correct Cases. Table 7 presents the performance of the proposed LLM-based workflow on the 69 clone cases where JDeodorant successfully applied Extract Method refactoring. Since these cases already satisfy JDeodorant’s strict structural preconditions, JDeodorant achieves perfect usefulness, compilation, and testing correctness across all cases. Among the LLM-based approaches, Gemini 3 Pro achieves the highest overall testing performance at 73.9%, followed by DeepSeek v3.2 at 71.0% and Claude Opus 4.6 at 65.2%. In contrast, GPT-5.2 achieves the lowest testing performance at 63.8%, despite maintaining relatively high detection and usefulness rates.

The results suggest that even for clone cases already considered safely refactorable by JDeodorant, generating behavior-preserving Extract Method transformations remains challenging for LLM-based workflows. While most models achieve near-perfect detection performance and high usefulness rates, substantial performance drops are still observed during compilation and testing valida-

Table 8: LLM Workflow Performance on No Refactoring Opportunities Cases

Model	Detected	Useful	Compilation	Test
DeepSeek v3.2	9 (90.0%)	8 (80.0%)	8 (80.0%)	8 (80.0%)
Gemini 3 Pro	9 (90.0%)	10 (100.0%)	10 (100.0%)	9 (90.0%)
Claude Opus 4.6	9 (90.0%)	10 (100.0%)	10 (100.0%)	10 (100.0%)
GPT-5.2	6 (60.0%)	8 (80.0%)	6 (60.0%)	6 (60.0%)

Notes: Results are computed on the 10 clone cases categorized by JDeodorant as *No Refactoring Opportunities*. Useful, compilation, or test counts may exceed detection counts because some cases not detected by the LLM-based detector were later recovered through the PSI-based fallback detection mechanism.

Table 9: LLM Workflow Performance on JDeodorant Precondition Violation Cases

Model	Detected	Useful	Compilation	Test
DeepSeek v3.2	40 (97.6%)	37 (90.2%)	34 (82.9%)	28 (68.3%)
Gemini 3 Pro	38 (92.7%)	36 (87.8%)	36 (87.8%)	31 (75.6%)
Claude Opus 4.6	40 (97.6%)	38 (92.7%)	34 (82.9%)	32 (78.0%)
GPT-5.2	26 (63.4%)	27 (65.9%)	21 (51.2%)	20 (48.8%)

Notes: Results are computed on the 41 clone cases that triggered JDeodorant precondition violations. Useful, compilation, or test counts may exceed detection counts because some cases not detected by the LLM-based detector were later recovered through the PSI-based fallback detection mechanism.

tion. This indicates that structurally reasonable refactorings generated by LLMs may still introduce unresolved dependencies, incorrect extraction boundaries, or behavioral inconsistencies. Overall, the comparison highlights the difference between conservative rule-based refactoring tools and flexible LLM-based generation approaches: JDeodorant prioritizes structural safety before applying transformations, whereas LLM-based workflows attempt broader refactoring generation but require additional validation stages to ensure correctness.

No Refactoring Opportunities. Table 8 presents the manual validation results for clone cases categorized by JDeodorant as *No Refactoring Opportunities*. The results show that most of these cases were still considered refactorable when evaluated using the outputs generated by the proposed LLM-based workflows. Gemini 3 Pro and Claude Opus 4.6 achieve the highest correctness rate at 90.0%, followed by DeepSeek v3.2 at 80.0%. In contrast, GPT-5.2 achieves a lower correctness rate of 60.0%, indicating comparatively weaker consistency in generating behavior-preserving and structurally valid Extract Method refactorings for structurally difficult clone instances.

Overall, the manual inspection suggests differences in the ability of LLMs to generate alternative Extract Method transformations for structurally challenging clones. In particular, Claude Opus 4.6 and Gemini 3 Pro demonstrate stronger capability in identifying practically refactorable code fragments.

Table 10: Common Strategies Used by LLMs to Handle JDeodorant Precondition Violations

Precondition Category	Violation	LLM Refactoring Strategy
Return/control-flow inconsistency		Preserved return statements and branch-specific logic in the original methods while extracting only the shared computation or duplicated statements into helper methods.
Type unification failure		Extracted common control-flow or validation logic while retaining type-specific operations and object accesses at the original call sites.
Dependency constraint violation		Performed partial extraction by moving only dependency-independent statements into helper methods and leaving dependency-sensitive expressions in the original methods.
Control dependency violation		Reorganized the refactoring using if-else structures or separated conditional branches to preserve the original execution semantics during extraction.
Void method parameterization issue		Introduced additional parameters, boolean flags, helper wrappers, or lambda-style dispatch logic to handle different void setter operations within extracted methods.
Incompatible return types		Kept type-specific return handling in the caller methods while extracting only the shared intermediate computation logic.
No matched statements		Identified higher-level semantic similarities and extracted surrounding duplicated logic instead of relying only on exact statement matching.

Precondition Violation. Table 9 presents the performance of the LLM-based workflow on the 41 clone cases that triggered JDeodorant precondition violations. The results show that Claude Opus 4.6 and DeepSeek v3.2 achieve the highest detection rate at 97.6%, followed closely by Gemini 3 Pro at 92.7%. Claude Opus 4.6 also achieves the highest usefulness rate at 92.7%, while Gemini 3 Pro achieves the strongest compilation and testing performance at 87.8% and 75.6%, respectively. In contrast, GPT-5.2 shows substantially lower performance across all evaluation stages, particularly for compilation and testing correctness.

These results suggest that many clone cases rejected by JDeodorant due to structural preconditions can still be successfully refactored using the proposed LLM-based workflow. In particular, the workflow is often able to generate behavior-preserving transformations by adjusting extraction boundaries, partially extracting shared logic, or restructuring dependency-sensitive statements.

Table 10 summarizes several common strategies used by LLM-based workflows to handle cases that triggered JDeodorant precondition violations. The

manual analysis shows that LLMs are often able to generate alternative Extract Method transformations by relaxing strict structural constraints and the duplicated code. Common strategies include modifying extraction boundaries, preserving return statements in the original methods, introducing additional parameters or wrapper methods, and partially extracting only the shared logic while leaving dependency-sensitive statements unchanged. These flexible transformations allow the generated refactorings to successfully compile and preserve behavioral correctness even when JDeodorant did not perform the refactoring due to precondition violations.

However, although these LLM-generated refactorings can bypass many precondition violations, the resulting code does not always follow high-quality refactoring design principles. In several cases, the generated helper methods introduce unnecessary parameters, boolean control flags, duplicated conditional logic, or overly complicated method structures in order to satisfy compilation and testing requirements. Compared to JDeodorant, which applies conservative structural analysis and only performs refactorings when clean extraction conditions are satisfied, LLM-based workflows tend to prioritize functional correctness over long-term maintainability and refactoring quality. This observation suggests that successfully compiling and passing tests does not necessarily imply that the resulting refactoring represents an ideal or maintainable design transformation.

These findings highlight an important difference between traditional refactoring tools and LLM-based approaches. JDeodorant focuses on preserving Extract Method refactoring quality, code elegance, and structural cleanliness through precondition checking, while LLM-based workflows can sometimes handle structurally difficult clone instances that traditional tools reject. As a result, LLMs can often refactor code fragments that traditional tools reject, but the generated transformations may sacrifice readability, simplicity, or maintainability in the process.

Example Challenging for Both JDeodorant and LLMs. Listing 24 and Figure 5 show an example clone pair that triggered the JDeodorant precondition violation message: “Expression (UseCase) `handle` cannot be unified with expression (Element) `handle`, because the common superclass does not declare the required members.” Although the two methods are structurally similar and both follow the same validation-and-return pattern, the returned object types belong to different class hierarchies. As a result, JDeodorant cannot safely parameterize or unify the expressions into a single Extract Method operation under its strict type unification rules.

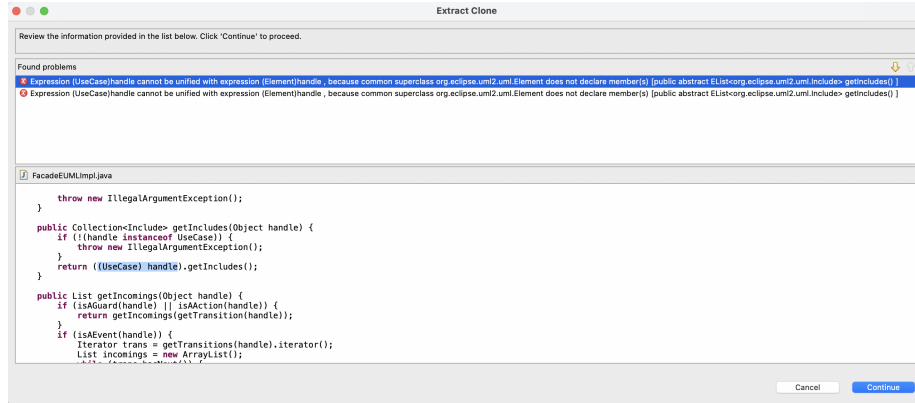


Figure 5: JDeodorant Precondition Violation.

Listing 24: Example Rejected by JDeodorant Due to Type Unification Precondition

```
public Collection<Include> getIncludes(Object handle) {
    if (!(handle instanceof UseCase)) {
        throw new IllegalArgumentException();
    }
    return ((UseCase) handle).getIncludes();
}

public List<Element> getModelElementContents(Object handle) {
    if (!(handle instanceof Element)) {
        throw new IllegalArgumentException();
    }
    return ((Element) handle).getOwnedElements();
}
```

In this case, all four evaluated LLMs, including Claude Opus 4.6, DeepSeek v3.2, Gemini 3 Pro, and GPT-5.2, also failed to generate a correct refactoring. The generated outputs either introduced invalid abstractions, produced uncompileable helper methods, or failed to preserve the original method semantics. This example highlights that some precondition violations remain fundamentally difficult even for LLM-based workflows, particularly when the cloned fragments require non-trivial type generalization across incompatible object hierarchies.

Example Refactorable by LLMs but Rejected by JDeodorant. Listing 25 and Figure 6 show an example clone pair that triggered the JDeodorant precondition violation message: “Not all possible execution flows end in a return statement.” Although JDeodorant rejected this Extract Method opportunity due to conservative control-flow analysis, DeepSeek v3.2, Claude Opus 4.6, and Gemini 3 Pro were all able to generate useful refactorings that successfully compiled and passed testing validation. Manual inspection showed that these models preserved the original control-flow behavior by restructuring the extracted logic

while maintaining the required return semantics in the caller methods. This example suggests that some clone cases rejected by traditional refactoring tools due to strict structural preconditions may still be practically refactorable using flexible LLM-generated transformations.

Listing 25: Example Rejected by JDeodorant Due to Return-Flow Precondition

```
public Object getDestination(Object trans) {
    if (trans instanceof Transition) {
        return ((Transition) trans).getTarget();
    }
    throw new IllegalArgumentException();
}

public Object getSource(Object trans) {
    if (trans instanceof Transition) {
        return ((Transition) trans).getTarget();
    }
    throw new IllegalArgumentException();
}
```

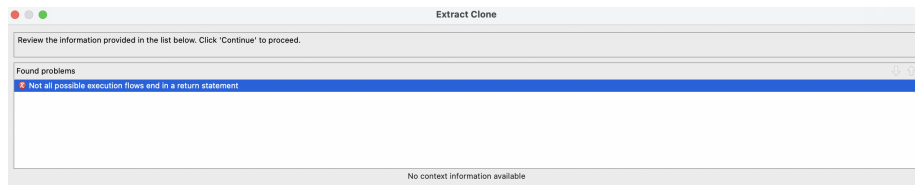


Figure 6: JDeodorant Precondition Violation 2.

RQ1 Summary.

On real-world clones, the LLM driven workflow maintains strong effectiveness, achieving a detection rate of 93.1% (447/480) and producing useful Extract Method refactorings in 432 out of 480 cases (90.0%). Claude Opus 4.6 and Gemini 3 Pro achieve the highest overall usefulness rate (112/120, 93.3%), followed by DeepSeek v3.2 (109/120, 90.8%), while GPT-5.2 shows lower performance (99/120, 82.5%).

The results further reveal that clone size influences refactoring effectiveness. Small and medium-sized clones show relatively similar performance, with medium clones (6–10 lines) achieving slightly higher usefulness (90.6%) while both groups obtain the same test pass rate (73.1%). In contrast, large clones (10+ lines) exhibit noticeably lower performance, with the test pass rate decreasing to 61.3%. This suggests that structural complexity and dependency interactions become more challenging as clone size increases.

The comparison with JDeodorant further highlights the trade-off between conservative rule-based refactoring and flexible LLM-based transformation generation. JDeodorant achieves perfect correctness on the 69 structurally valid cases where all extraction preconditions are satisfied, but rejects 51 out of 120 real-world clone cases due to strict precondition constraints. In contrast, the proposed workflow is able to generate useful and behavior-preserving refactorings for many of these rejected cases, particularly for clones involving return-flow inconsistencies, dependency constraints, and partial extraction opportunities. However, the generated refactorings do not always preserve ideal maintainability or structural simplicity, since some outputs introduce additional parameters, control flags, or more complex helper methods in order to satisfy compilation and testing requirements.

Overall, the workflow generalizes well to real-world scenarios, but its effectiveness is influenced by clone size.

5.2. RQ2: What is the contribution of each component of the LLM driven framework?

5.2.1. Approach

To answer RQ2, we conduct an ablation study to understand the contribution of each major component in our LLM driven framework. The goal is to isolate the effect of individual components and measure how much each contributes to the final refactoring success rate.

We use the same 120 clone cases on the ArgoUML project described in RQ1. We use Deepseek v3.2 because the results from RQ1 show that its overall performance is relatively close to the top-performing models, while requiring substantially lower inference cost and execution time. This makes Deepseek v3.2 a practical and cost-effective choice for conducting extensive ablation experiments involving multiple workflow configurations and iterative executions.

Table 11: Ablation Study Results on 120 Real-World Clone Cases (RQ2)

Setting	Detected	Useful	Compile	Test	Success
Full Workflow	117 (97.5%)	109 (90.8%)	96 (80.0%)	84 (70.0%)	84 (70.0%)
No RAG	120 (100.0%)	99 (82.5%)	84 (70.0%)	37 (30.8%)	37 (30.8%)
No Usefulness Checker	119 (99.2%)	111 (92.5%)	87 (72.5%)	77 (64.2%)	77 (64.2%)
No Feedback Loop	119 (99.2%)	105 (87.5%)	94 (78.3%)	77 (64.2%)	77 (64.2%)
No Panelists	119 (99.2%)	99 (82.5%)	85 (70.8%)	77 (64.2%)	77 (64.2%)

Note. Each ablation setting is evaluated on the same 120 real-world clone cases using DeepSeek v3.2.

The dataset, evaluation criteria, and maximum number of attempts remain the same. Only the workflow configuration is changed.

We design four ablation settings:

(1) **No RAG:** We remove retrieval-augmented generation from both the Detection Agent and the Refactoring Agent. The models no longer receive example-based guidance in the prompt.

(2) **No Usefulness Checker:** We disable the usefulness checker. Refactorings are evaluated only by compilation and testing, without structural validity checking.

(3) **No Feedback Loop:** We remove the iterative feedback mechanism. Each model generates a refactoring only once, without receiving compile errors, test failures, or usefulness feedback for correction.

(4) **No Panelists:** We remove the panelist and curator architecture from the workflow. Instead of using multiple collaborative agents for detection, refactoring, and usefulness evaluation, each stage is performed by a single LLM without multi-agent discussion or aggregation. This setting evaluates the contribution of collaborative reasoning and cross-agent validation in the workflow.

For each setting, we run DeepSeek v3.2 in the same 120 clone cases. We measure detection rate, compilation success, test passing rate, and final usefulness-based success when applicable. By comparing these ablated variants with the full LLM driven workflow, we can quantify how much each component contributes to improving structural correctness and overall refactoring success.

5.2.2. Results

No RAG. Table 11 shows the ablation result after removing the RAG component from the workflow. The detection result remains stable, with the No RAG setting detecting all 120 clone cases (100.0%). This suggests that retrieval examples are not essential for clone localization in this setting, since the Detection Agent can still identify duplicated regions without example-based guidance.

However, the downstream refactoring performance drops substantially after removing RAG. Compared with the full workflow, where DeepSeek v3.2 achieves 109 useful refactorings (90.8%), 96 compilable refactorings (80.0%), and 84 successful test-passing refactorings (70.0%), the No RAG setting produces only 99 useful refactorings (82.5%), 84 compilable refactorings (70.0%), and 37 successful test-passing refactorings (30.8%). The most significant decrease appears in

the testing stage, where the success rate drops by 33.4 percentage points.

These results indicate that RAG contributes mainly to the quality and behavioral stability of the generated refactorings rather than to clone detection. Without retrieved examples, the model can still locate clones and generate refactorings that often appear structurally useful, but many of these refactorings fail to preserve the original behavior under testing. This suggests that example-based guidance helps the model follow more appropriate Extract Method patterns, maintain correct extraction boundaries, and reduce behavior-changing transformations.

Overall, the No RAG ablation shows that retrieval plays an important stabilizing role in the refactoring stage. While it does not noticeably improve detection, it helps the workflow generate refactorings that are more likely to compile and pass behavioral validation.

No Usefulness Checker. Table 11 presents the ablation result after removing the usefulness checker from the workflow. The first observation is that the detection performance remains nearly unchanged. The No Usefulness Checker setting still detects 119 out of 120 clone cases (99.2%), which is very close to the full workflow. This indicates that the usefulness checker does not contribute to clone localization, but instead affects the quality control process after refactoring generation.

However, removing the usefulness checker noticeably reduces the downstream refactoring quality. Compared with the full workflow, where DeepSeek v3.2 achieves 96 compilable refactorings (80.0%) and 84 successful test-passing refactorings (70.0%), the No Usefulness Checker setting produces only 87 compilable refactorings (72.5%) while maintaining the same final testing success rate of 64.2%. In addition, because the usefulness checker is disabled, the workflow no longer filters structurally weak or incomplete Extract Method operations before compilation and testing.

An important observation is that the reduction caused by removing the usefulness checker is larger than the reduction observed in the No Feedback Loop setting for compilation-related correctness. This suggests that the usefulness checker plays an important role in eliminating structurally invalid refactorings early in the workflow. Without this validation stage, the system accepts more refactorings that appear syntactically correct but do not fully satisfy the intended Extract Method structure.

Overall, these results indicate that the usefulness checker is a major contributor to structural correctness in the workflow. While compilation and testing can verify syntactic and behavioral validity, they are insufficient for ensuring that the generated transformation truly performs a proper Extract Method refactoring. The usefulness checker, therefore, serves as an important structural validation layer beyond traditional compile- and test-based verification.

No Feedback Loop. Table 11 presents the ablation result after removing the feedback loop from the workflow. Similar to the other ablation settings, the detection performance remains nearly unchanged. The No Feedback Loop setting

still detects 119 out of 120 clone cases (99.2%), which is very close to the full workflow. This indicates that the feedback loop mainly affects the refactoring refinement stage rather than the clone localization stage.

The largest changes appear in the downstream refactoring quality. Compared with the full workflow, where DeepSeek v3.2 achieves 109 useful refactorings (90.8%), 96 compilable refactorings (80.0%), and 84 successful test-passing refactorings (70.0%), the No Feedback Loop setting produces only 105 useful refactorings (87.5%) and 94 compilable refactorings (78.3%). However, the final testing success rate remains unchanged at 64.2%.

These results suggest that the feedback loop primarily improves structural stability and refinement quality during refactoring generation. In the normal workflow, the feedback loop allows the model to revise structurally weak or incomplete Extract Method operations after receiving validation feedback. Without this iterative correction stage, structurally problematic initial generations are directly treated as final outputs. This explains the reduction in usefulness and compilation correctness after removing the feedback loop.

Compared with the No Usefulness Checker setting, the impact of removing the feedback loop is smaller, indicating that iterative refinement alone is insufficient without structural validation. Nevertheless, the feedback loop still contributes to workflow stability by improving borderline refactorings before compilation and testing. Overall, these results show that iterative correction is an important supporting mechanism for improving the structural quality of generated Extract Method refactorings.

No Panelists and Curator. Table 11 presents the ablation result after removing the panelist and curator architecture from the workflow. Similar to the other ablation settings, the detection performance remains stable. The No Panelists setting still detects 119 out of 120 clone cases (99.2%), which is very close to the full workflow. This indicates that clone localization itself is not strongly dependent on collaborative LLM driven reasoning.

However, removing the panelist and curator structure leads to noticeable reductions in downstream refactoring quality. Compared with the full workflow, where DeepSeek v3.2 achieves 109 useful refactorings (90.8%), 96 compilable refactorings (80.0%), and 84 successful test-passing refactorings (70.0%), the No Panelists setting produces only 99 useful refactorings (82.5%) and 85 compilable refactorings (70.8%). Although the final testing success rate remains unchanged at 64.2%, the decrease in usefulness and compilation correctness suggests that single-agent generation is less stable when handling structurally difficult clone cases.

These results suggest that the panelist and curator architecture mainly contribute to structural reasoning and quality control during refactoring generation. In the full workflow, multiple agents collaboratively evaluate candidate outputs for detection, refactoring, and usefulness validation before aggregation by the curator. This process helps reduce structurally weak, incomplete, or inconsistent Extract Method transformations. Without cross-agent discussion and validation, the workflow becomes more dependent on the quality of a single

generation, increasing the likelihood of structurally problematic refactorings.

Compared with the No Feedback Loop setting, the reduction caused by removing the panelist architecture is larger, especially for usefulness and compilation correctness. This indicates that collaborative reasoning contributes more strongly to structural correctness than iterative refinement alone. Overall, these findings suggest that LLM driven collaboration plays an important role in improving the reliability and structural quality of LLM-based Extract Method refactoring.

Overall Analysis of RQ2. Table 11 and Figure 7 provides an overall comparison of all ablation settings in RQ2. One observation is immediately clear across all settings: the detection performance remains highly stable, while usefulness, compilation correctness, and testing success vary substantially after removing structural support components. This again suggests that clone localization itself is not the primary challenge in LLM-based clone refactoring. Instead, the main difficulty lies in generating structurally correct and behavior-preserving Extract Method transformations.

Among all settings, the full workflow achieves the strongest overall performance, producing 109 useful refactorings (90.8%), 96 compilable refactorings (80.0%), and 84 successful test-passing refactorings (70.0%). In contrast, removing RAG causes the largest decrease in behavioral correctness, reducing the final testing success rate from 64.2% to 30.8%. Although the No RAG setting still detects all 120 clone cases and produces 99 structurally useful refactorings

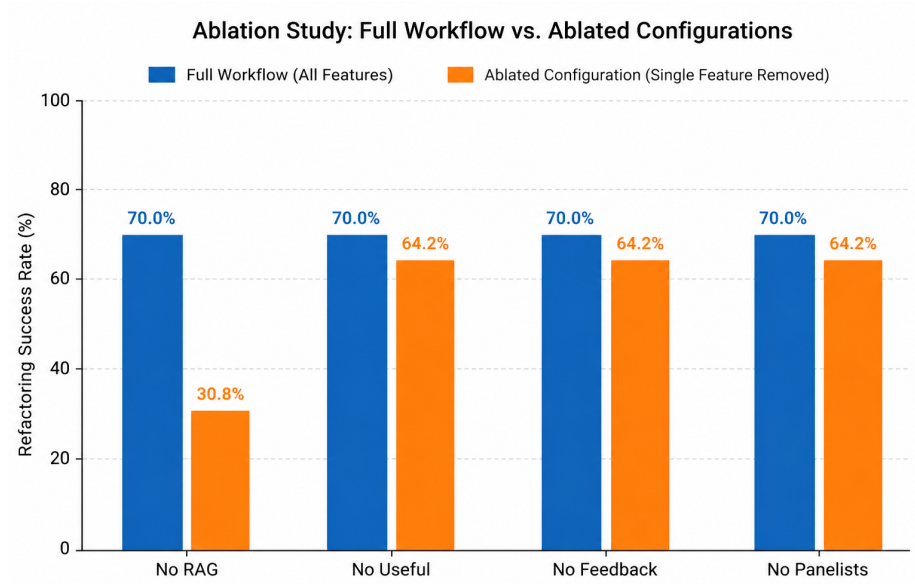


Figure 7: RQ2 ablation study success rate with DeepSeek v3.2.

(82.5%), many generated refactorings fail during behavioral validation. This indicates that retrieval guidance mainly contributes to preserving behavioral correctness and stabilizing extraction patterns during refactoring generation.

The removal of the usefulness checker and the panelist architecture mainly affects structural correctness. Compared with the full workflow, both settings produce noticeable reductions in usefulness and compilation correctness, even though their final testing success rates remain unchanged. This suggests that compilation and testing alone are insufficient for evaluating Extract Method refactoring quality. Many generated transformations may still compile and preserve behavior while failing to satisfy the intended structural properties of proper Extract Method refactoring.

Another important observation is that removing the feedback loop produces a smaller reduction than removing the usefulness checker or collaborative LLM driven reasoning. This suggests that iterative refinement is helpful for correcting borderline generations, but structural validation remains the dominant factor in ensuring reliable refactoring quality.

Overall, the results of RQ2 demonstrate that structural enforcement mechanisms play a critical role in LLM-based clone refactoring workflows. Detection alone is insufficient, and behavioral validation alone cannot guarantee structurally valid Extract Method transformations. High-quality refactoring results can only be achieved through the combined contributions of retrieval guidance, structural usefulness validation, collaborative reasoning, and iterative refinement.

RQ2 Summary.

Across all ablation settings, the detection performance remains highly stable, with all configurations detecting either 119 or 120 out of 120 clone cases. This suggests that clone localization itself is not the primary challenge in LLM-based clone refactoring. Instead, the major differences appear in the structural correctness and behavioral stability of the generated Extract Method transformations.

Among all components, removing RAG produces the largest reduction in behavioral correctness. Although the No RAG setting still detects all 120 clone cases and achieves 99 useful refactorings (82.5%), the final testing success rate drops substantially from 64.2% to 30.8%. This indicates that retrieval guidance mainly contributes to preserving behavioral correctness and stabilizing extraction patterns during refactoring generation.

The usefulness checker and collaborative LLM driven reasoning mainly improve structural correctness. Removing either component leads to noticeable reductions in usefulness and compilation correctness, even when the final testing success rate remains unchanged. This suggests that compilation and testing alone are insufficient for evaluating Extract Method refactoring quality, since many generated transformations can still compile and preserve behavior while failing to satisfy the intended structural properties of proper Extract Method refactoring.

Compared with the usefulness checker and panelist architecture, removing the feedback loop produces a smaller reduction in downstream performance. This suggests that iterative refinement mainly helps repair borderline generations, while structural validation remains the dominant factor in ensuring reliable refactoring quality. Overall, the results demonstrate that high-quality clone refactoring depends on the combined contributions of retrieval guidance, structural usefulness validation, collaborative reasoning, and iterative refinement.

6. Discussion

6.1. Structural Correctness vs. Behavioral Correctness

The results reveal that there is a significant difference between behavioral correctness and structural correctness. In the single LLM case, all generated refactoring code compiles correctly and passes the test cases, but only 70.4% of the code meets the Extract Method definition. This implies that it is not guaranteed that the code behavior is preserved to correctly remove the code clones.

In some cases, the generated code preserves the program’s functionality but fails to correctly perform the Extract Method refactoring. For example, refactorings such as direct code clone removal or incomplete refactoring can be syntactically correct and pass all the test cases, but they have not achieved the goal of the Extract Method refactoring.

The observation implies that automated refactoring systems should not be evaluated solely by code compilation correctness and test-case pass rates. The code behavior has been correctly preserved in the refactoring operation, but the actual goal of the refactoring operation may not be achieved correctly. The refactoring structure correctness should be explicitly evaluated by the defined criteria.

The difference also implies that producing syntactically correct code alone is not sufficient for refactoring. The tests’ passing primarily assures behavioral preservation, whereas the Extract Method refactoring needs more attention regarding the structure of the code. A possible solution is to incorporate some structure constraints to verify if the refactoring satisfies the requirement of the Extract Method.

6.2. Component Interaction and Dependency

The ablation study in RQ2 demonstrates that reliable clone refactoring depends on the interaction of multiple workflow components rather than a single generation mechanism. Although all ablation settings maintain highly stable detection performance, noticeable differences appear in usefulness, compilation correctness, and behavioral correctness after removing different structural support components. This suggests that the main difficulty lies in generating structurally correct and behavior-preserving Extract Method transformations.

Among all settings, the full workflow achieves the strongest overall performance, producing 109 useful refactorings (90.8%), 96 compilable refactorings (80.0%), and 84 successful test-passing refactorings (70.0%). In contrast, removing RAG causes the largest reduction in behavioral correctness, reducing the final testing success rate from 70.0% to 30.8%. Although the No RAG setting still detects all 120 clone cases and produces 99 structurally useful refactorings (82.5%), many generated refactorings fail during testing validation. This indicates that retrieval guidance mainly contributes to preserving behavioral correctness and stabilizing extraction patterns during refactoring generation. The retrieved examples help the model generate more appropriate extraction boundaries and reduce behavior-changing transformations during the initial generation stage.

The usefulness checker and collaborative LLM driven reasoning mainly contribute to structural correctness. Compared with the full workflow, removing the usefulness checker reduces compilation correctness from 80.0% to 72.5%, while removing the panelist architecture reduces it further to 70.8%. At the same time, usefulness decreases from 90.8% to 82.5% in the No Panelists setting. These results suggest that compilation and testing alone are insufficient for evaluating Extract Method refactoring quality. Many generated transformations may still compile and preserve behavior while failing to satisfy the intended structural properties of proper Extract Method refactoring. The usefulness checker, therefore, serves as an important structural validation layer beyond traditional behavioral verification.

Compared with the usefulness checker and collaborative reasoning structure, removing the feedback loop produces a smaller reduction in downstream per-

formance. The No Feedback Loop setting still achieves 105 useful refactorings (87.5%) and 94 compilable refactorings (78.3%), suggesting that iterative refinement mainly helps repair borderline generations rather than fundamentally changing refactoring quality. In practice, the feedback loop is particularly useful for correcting incomplete extraction, unstable extraction boundaries, and partially duplicated code that survive the initial generation stage.

Overall, the results indicate that high-quality clone refactoring cannot be achieved through prompting alone. Instead, reliable Extract Method refactoring requires coordinated support for generation, structural validation, behavioral verification, and iterative refinement. The study, therefore, suggests that future research should focus more on workflow-level coordination and structural enforcement mechanisms rather than relying solely on larger models or stronger prompting strategies.

6.3. Effect of Clone Size on Refactoring Stability.

Clone size influences the difficulty of Extract Method refactoring in a different way than clone type complexity. Medium-sized clones appear to provide the most balanced setting for the workflow because they usually contain enough duplicated context to make the extraction target explicit, while still remaining relatively localized and structurally manageable. In contrast, very small clones often contain only a few duplicated statements, making it unclear whether a meaningful abstraction should be introduced at all. This increases the risk of generating unnecessary helper methods or selecting unstable extraction boundaries. Large clones introduce the opposite problem. Although the duplicated structure is easier to recognize, the surrounding context becomes substantially more complex, including nested control flow, variable dependencies, and interactions with external objects or methods. In these cases, the challenge is no longer simply detecting duplication, but preserving behavior while reorganizing a larger portion of the program structure. The results, therefore, suggest that successful clone refactoring depends not only on identifying similar code regions but also on balancing contextual completeness with structural manageability during extraction.

6.4. Consideration of Type-4 Clones

Type-4 clones are semantically similar, but syntactically different from each other. Because of this, structural or token-based similarity is generally not sufficient to detect Type-4 clones.

While our experimental results in RQ1 and RQ2 indicate that structural enforcement is generally insufficient, especially when syntax differs significantly for Type-4 clones, the issue is no longer how to identify duplicated structure, but rather how to identify duplicated behavior. This might be achieved by employing several semantic analysis strategies, such as program dependence graphs, program execution graphs, or embedding-based similarity analysis, to identify duplicated behavior in the code.

In addition, refactoring Type-4 clones would require a higher level of behavioral guarantees, as the extraction boundary might no longer be clear, and

behavioral preservation is now more important. This implies that test coverage must now be sufficient to cover the shared behavior, and possibly additional constraints must now be imposed on the generation process, such as data dependency preservation and side effect preservation from both code fragments.

Moreover, the usefulness checker would also have to be adapted. Currently, our usefulness checker is based on structural information. For Type-4 clones, structural similarity may already be low. Therefore, it could be necessary to extend the usefulness evaluation to semantic consolidation instead of token reduction. This could be achieved by checking if both original code fragments refer to a common abstraction that expresses the same steps, even if these steps are represented by different internal statements.

We do not claim that our current workflow is able to deal with Type-4 clones directly. However, we hope that the results of this study can be used as a basis for design decisions for an extension of existing clone detection tools to Type-4 clones. This would require a more powerful semantic analysis and additional evaluation tools, which is left for future work.

6.5. Code Quality Considerations

Our evaluation criteria include structural correctness, behavioral preservation, and usefulness according to the definition of the Extract Method technique. However, we are not measuring other code quality attributes such as complexity, cohesion, coupling, readability, and maintainability.

It is possible that two different refactorings may pass all tests and be correct according to structural criteria but differ in code quality attributes. For example, a function may be extracted and be correct according to our validation criteria, but still be a bad function because it has too many parameters, it has reduced cohesion, or it has increased coupling with other classes.

As a possible extension of this work, it could be interesting to include existing code quality analysis tools to measure code quality after the refactoring. For example, tools such as Understand or other software metrics analyzers could be used to measure complexity, dependency graphs, and cohesion before and after the refactoring process.

In addition, quality optimization itself might be a role that is added to the workflow explicitly, in the form of a quality-aware agent. The agent might analyze changes in metrics and provide feedback on how to improve the code further. It is worth noting that this agent would not replace structural or behavioral validation, but rather complement it, focusing on design improvements.

While we are not claiming that the current workflow improves overall quality better than duplication reduction, we are establishing a base that is centered around structural validity and correctness, and that improving the workflow in a multi-dimensional quality optimization is a significant area for future research.

6.6. Expanding Clone Detection and Refactoring Across Files

One limitation with our current workflow is that it only looks for clones within the current file. However, in real software projects, duplication can often occur across multiple files rather than within a single file.

Addressing this limitation is important because supporting cross-file clone detection would significantly increase the impact of our tool. Clones across different files can be even harder for developers to detect, increasing maintenance issues and inconsistencies over time. Clones within the same file are usually easy to spot, but when code is duplicated across files, there is no easy way for a developer to detect them without explicitly searching. The larger and more complicated the project becomes, the more valuable this becomes as the chances of duplicating logic across different files only increase with size and complexity. This is the kind of problem our tool aims to prevent, so extending it to detect clones across files would enable it to catch duplications that, in most cases, would otherwise go unaddressed.

The foundation established in this study provides a good starting point for this extension. Future work could build on this workflow to support cross-file detection and refactoring, and evaluate whether it retains the same improvements in usefulness and correctness at a larger scale. Although this extension was not included in the current version of the tool, it represents an important direction for future work in making the workflow more practical for real-world development.

6.7. Key Insights

6.7.1. Detection Agents Require Deterministic Grounding

The RQ1 detection results show why deterministic grounding is necessary in LLM-based clone detection. Although the workflow achieves a high overall detection rate of 93.1% (447/480), the models do not produce equally reliable clone localization results. For example, GPT-5.2 detects only 97 out of 120 cases directly, and 16 additional cases are recovered through the PSI-based fallback mechanism. This suggests that LLM-based detection can miss or inaccurately localize clone ranges even when the target duplication is present. Therefore, future systems should not directly trust LLM-reported ranges. Instead, detected snippets should be grounded back to the original source code using text matching, range repair, and PSI-based validation before being passed to the refactoring stage.

6.7.2. Usefulness Evaluation Requires Both Semantic and Structural Validation

The RQ1 results show that usefulness, compilation, and testing capture different aspects of refactoring correctness. Across 480 evaluated runs, the workflow produces 432 useful refactorings (90.0%), but only 384 compile successfully (80.0%) and 332 pass testing (69.2%). This gap suggests that structural usefulness alone is not sufficient to guarantee syntactic or behavioral correctness. At the same time, the RQ2 ablation shows that removing the usefulness checker reduces compilation success from 96/120 (80.0%) to 87/120 (72.5%), even though detection remains nearly unchanged. Therefore, usefulness evaluation should combine structural validation with compilation and behavioral testing rather than relying on any single validation signal.

6.7.3. *Candidate Diversity Alone Is Insufficient*

The RQ2 ablation results show that candidate diversity alone is insufficient for ensuring correct refactoring outcomes. When the panelist and curator stages are removed, the final test success rate drops from 84/120 (70.0%) to 77/120 (64.2%), suggesting that collaborative candidate generation and selection improve downstream refactoring quality. However, even in the full workflow, there remains a noticeable gap between usefulness (109/120, 90.8%) and final test success (84/120, 70.0%). This indicates that selecting among multiple candidate refactorings can improve the likelihood of generating useful Extract Method transformations, but it does not automatically resolve syntax errors, dependency issues, or behavioral inconsistencies. Therefore, curator selection should not be treated as the final validation stage. Instead, candidate diversity should be combined with strict compilation and behavioral testing gates before accepting a refactoring.

6.7.4. *LLMs Do Not Always Follow Refactoring Guidelines Reliably*

Another important observation from the study is that LLMs do not always reliably follow explicit workflow guidelines during either clone detection or refactoring generation. In the detection stage, models occasionally report incorrect clone boundaries, partially overlapping regions, or line numbers that do not exactly correspond to the original source code, even when the prompt explicitly requests exact clone ranges. In several cases, the models also returned structurally similar regions that were not true Extract Method candidates according to the intended refactoring objective. These observations suggest that prompt instructions alone are insufficient for ensuring stable and deterministic clone localization.

A similar issue appears during refactoring generation. Although the prompts explicitly restrict the models to perform only Extract Method refactoring with minimal modifications, some generated outputs still introduce unrelated structural changes, partially remove clones without proper extraction, or modify surrounding logic beyond the duplicated fragment. In other cases, the generated refactorings compile successfully while still violating the intended structural properties of the Extract Method refactoring. These findings further support the need for lightweight structural validation, compilation gates, testing validation, and iterative feedback rather than relying solely on prompt compliance.

Overall, the results indicate that modern LLMs can often follow high-level refactoring instructions, but their adherence to detailed structural constraints remains inconsistent. Therefore, future LLM-based refactoring systems should avoid assuming perfect instruction following behavior and instead incorporate additional grounding, validation, and repair mechanisms throughout the workflow.

6.7.5. *More Agents Do Not Always Improve Workflow Quality*

Finally, our design process shows that adding more agents does not always make the workflow better. A fully panelist-based design across detection, refac-

toring, and usefulness can increase coverage and usefulness, but it also introduces new sources of instability, higher cost, and more dependence on curator decisions. The main observation is that agents should be added only when they address a concrete failure mode. Detection panelists help with missed clones, refactoring panelists help with candidate diversity, usefulness panelists help with diagnostic feedback, and lightweight PSI fallback/checkers provide additional structural constraints. For future researchers, the key design principle is not simply to increase the number of agents, but to combine LLM reasoning with program analysis so that each component compensates for a specific weakness of the others.

7. Threats to Validity

Internal validity. One potential threat comes from how usefulness is evaluated. In this study, usefulness is approximated using rule-based structural checks, such as clone similarity reduction and PSI-level evidence of Extract Method refactoring. These checks are motivated by failure patterns observed in the empirical study [30]. However, they still reflect specific assumptions about what constitutes a meaningful Extract Method refactoring. It is possible that some refactorings marked as not useful by our checker would be acceptable to developers in certain contexts.

Another internal threat comes from the test coverage of the EvoSuite tests. In the proposed workflow, testing is used as feedback for the preservation of behavior after refactoring. However, the tests, although automated, may not completely cover the cloned methods. Out of 120 clone cases, 79 of them have test coverage that directly tests the cloned methods. For the remaining cases where the cloned methods are not tested, we manually generate the test file using GPT-5.5 and test code after refactoring to ensure that behavior is preserved. Although this reduces the threat of semantic changes, the test coverage may influence the validity of the testing feedback used for evaluation.

External validity. The evaluation of the workflow is restricted to Java-based projects, while only one type of refactoring Extract Method is used for evaluation purposes. The outcomes of the evaluation may not be applicable to other programming languages as well as other types of refactoring operations. Besides, the workflow is initiated by paste operations, while clones within one file are used for analysis purposes. This approach is based on the interactive IDE scenario, which may not account for scenarios where clones emerge over time across files.

Another threat to the study’s external validity is associated with the selection of the clone cases used for evaluation purposes. This work is designed as a case study based on the ArgoUML project for Extract Method refactoring evaluation. The 120 clone groups used in the study are derived from the ArgoUML project for Extract Method refactoring evaluation. However, all evaluation cases are collected from only one software project. Although the selected clone groups cover different clone structures and refactoring situations, they may not fully

capture the diversity, contextual complexity, and project-specific characteristics of real-world clones across different software systems.

The concept of usefulness adopted in this study is related to the concept of Extract Method refactoring. Different teams or projects can have different criteria for determining the usefulness of the refactoring. For instance, different people can have different criteria for refactoring. To minimize the subjective nature of the refactoring, the adopted criteria for determining the usefulness of the refactoring are based on the failure modes or refactoring patterns identified in the empirical study [30], such as incomplete refactoring, excessive refactoring, or logic fragmentation. However, it is accepted that different definitions of the concept of usefulness can exist, and the use of different criteria can yield different results.

8. Conclusion

This paper studies whether a structured LLM driven workflow can improve the reliability of LLM-based clone refactoring. We focus on Extract Method refactoring as the target operation.

The results show that the proposed workflow is effective on 120 real-world clone groups from ArgoUML. Overall, the workflow achieves 93.1% detection, 90.0% usefulness, 80.0% compilation success, and 69.2% test success across 480 model-level evaluations. These results suggest that the workflow can detect most clone instances and generate a large number of useful Extract Method refactorings, although preserving behavior remains more challenging than clone detection or structural usefulness.

The ablation study further shows that different components contribute to different aspects of refactoring quality. Removing RAG causes the largest drop in behavioral correctness, reducing the test success rate from 70.0% to 30.8%. Removing the usefulness checker or the panelist-curator structure mainly affects structural correctness and compilation reliability. These findings suggest that compilation and testing alone are not sufficient for evaluating Extract Method refactoring quality.

Overall, the findings from this case study suggest that prompting alone is often insufficient for producing reliable Extract Method refactorings in the evaluated workflow setting. The results further suggest that retrieval guidance, structural usefulness validation, behavioral testing, and multi-agent coordination can help improve the stability and correctness of generated Extract Method refactorings.

9. Acknowledgment

Declaration of generative AI and AI-assisted technologies in the writing process. During the preparation of this work, the first author used ChatGPT in order to facilitate brainstorming for research directions and improve the linguistic readability and flow of the manuscript, and assist in the implementation of the architecture, specifically for the usefulness checker, and bug

fixes. AI was further utilized for prompt refinement and generating automated instructions for compilation and test execution. Also, it is used to support the conceptualization and generation of algorithms presented in the study. After using these tools/services, the first author reviewed and edited the content as needed and takes full responsibility for the content of the publication, including the technical correctness of the code.

References

- [1] 2026. Replication Package for Multi-agent LLM-based Framework for Code Clone Refactoring. https://byte-shadow37.github.io/multi_agent_replication_package/. Accessed: 2026-03-05.
- [2] Ajmain I Alam, Palash R Roy, Farouq Al-Omari, Chanchal K Roy, Banani Roy, and Kevin A Schneider. 2023. GPTCloneBench: A comprehensive benchmark of semantic clones and cross-language clones using GPT-3 model and SemanticCloneBench. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 1–13.
- [3] Eman Abdullah AlOmar, Jacob Ashkenas, Robert Feliciano, Matthew Angelakos, Dimitrios Haralampopoulos, Xing Qian, Mohamed Wiem Mkaouer, and Ali Ouni. 2025. AntiCopyPaster 3.0: Just-in-Time Clone Refactoring. *ACM Transactions on Software Engineering and Methodology* (2025).
- [4] Eman Abdullah AlOmar, Anton Ivanov, Zarina Kurbatova, Yaroslav Golubev, Mohamed Wiem Mkaouer, Ali Ouni, Timofey Bryksin, Le Nguyen, Amit Kini, and Aditya Thakur. 2022. AntiCopyPaster: extracting code duplicates as soon as they are introduced in the IDE. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–4.
- [5] Eman Abdullah AlOmar, Anton Ivanov, Zarina Kurbatova, Yaroslav Golubev, Mohamed Wiem Mkaouer, Ali Ouni, Timofey Bryksin, Le Nguyen, Amit Kini, and Aditya Thakur. 2023. Just-in-time code duplicates extraction. *Information and Software Technology* 158 (2023), 107169.
- [6] Eman Abdullah AlOmar, Benjamin Knobloch, Thomas Kain, Christopher Kalish, Mohamed Wiem Mkaouer, and Ali Ouni. 2024. AntiCopyPaster 2.0: Whitebox just-in-time code duplicates extraction. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 84–88.
- [7] Eman Abdullah AlOmar, Mohamed Wiem Mkaouer, and Ali Ouni. 2024. Behind the intent of extract method refactoring: A systematic literature review. *IEEE Transactions on Software Engineering* 50, 4 (2024), 668–694.

- [8] Eman Abdullah AlOmar, Luo Xu, Sofia Martinez, Anthony Peruma, Mohamed Wiem Mkaouer, Christian D Newman, and Ali Ouni. 2025. ChatGPT for code refactoring: Analyzing topics, interaction, and effective prompts. In *2025 IEEE International Conference on Collaborative Advances in Software and COmputiNg (CASCOn)*. IEEE, 389–398.
- [9] Lerina Aversano, Luigi Cerulo, and Massimiliano Di Penta. 2007. How Clones are Maintained: An Empirical Study. In *11th European Conference on Software Maintenance and Reengineering (CSMR)*. 81–90. <https://doi.org/10.1109/CSMR.2007.26>
- [10] Abhiram Bellur, Mohammed Raihan Ullah, Fraol Batole, Mohit Kansara, Masaharu Morimoto, Kai Ishikawa, Haifeng Chen, Yaroslav Zharov, Timofey Bryksin, Tien N Nguyen, et al. 2026. Multi-Agent Coordinated Rename Refactoring. *arXiv preprint arXiv:2601.00482* (2026).
- [11] Jonathan Cordeiro, Shayan Noei, and Ying Zou. 2024. An empirical study on the code refactoring capability of large language models. *arXiv preprint arXiv:2411.02320* (2024).
- [12] Jonathan Cordeiro, Shayan Noei, and Ying Zou. 2025. LLM-Driven Code Refactoring: Opportunities and Limitations. In *2025 IEEE/ACM Second IDE Workshop (IDE)*. IEEE, 32–36.
- [13] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 758–759.
- [14] Marcus Vinicius Couto, Marco Tulio Valente, and Eduardo Figueiredo. 2011. Extracting Software Product Lines: A Case Study Using Conditional Compilation. In *15th European Conference on Software Maintenance and Reengineering (CSMR)*. 191–200.
- [15] Kayla DePalma, Izabel Miminoshvili, Chiara Henselder, Kate Moss, and Eman Abdullah AlOmar. 2024. Exploring ChatGPT’s code refactoring capabilities: An empirical study. *Expert Systems with Applications* 249 (2024), 123602.
- [16] Shihan Dou, Junjie Shan, Haoxiang Jia, Wenhao Deng, Zhiheng Xi, Wei He, Yueming Wu, Tao Gui, Yang Liu, and Xuanjing Huang. 2023. Towards understanding the capability of large language models on code clone detection: A survey. *arXiv preprint arXiv:2308.01191* (2023).
- [17] Martin Fowler. 2018. *Refactoring: improving the design of existing code*. Addison-Wesley Professional.
- [18] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM*

SIGSOFT symposium and the 13th European conference on Foundations of software engineering. 416–419.

- [19] Bowen Jiang, Mitchell Ruffolo, Aaditya Kulkarni, Teia Canini, and Eman Abdullah AlOmar. 2026. A systematic mapping study on the research landscape of LLM-based code clone detection. *Information and Software Technology* (2026), 108096.
- [20] Mohamad Khajezade, Jie JW Wu, Fatemeh Hendijani Fard, Gema Rodríguez-Pérez, and Mohamed Sami Shehata. 2024. Investigating the efficacy of large language models for code clone detection. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 161–165.
- [21] Miryung Kim, Vibha Sazawal, David Notkin, and Gail Murphy. 2005. An empirical study of code clone genealogies. In *Proceedings of the 10th European software engineering conference held jointly with 13th ACM SIGSOFT international symposium on Foundations of software engineering*. 187–196.
- [22] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: an empirical study. *Automated Software Engineering* 32, 1 (2025), 26.
- [23] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah Alomar. 2025. Software Refactoring Research with Large Language Models: A Systematic Literature Review. *Journal of Systems and Software* (2025), 112762.
- [24] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah Alomar. 2026. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software* 235 (2026), 112762.
- [25] Emerson Murphy-Hill, Chris Parnin, and Andrew P Black. 2011. How we refactor, and how we know it. *IEEE Transactions on Software Engineering* 38, 1 (2011), 5–18.
- [26] Suphakit Niwattanakul, Jatsada Singthongchai, Ekkachai Naenudorn, and Supachanun Wanapu. 2013. Using of Jaccard coefficient for keywords similarity. In *Proceedings of the international multiconference of engineers and computer scientists*, Vol. 1. 380–384.
- [27] Khouloud Oueslati, Maxime Lamothe, and Foutse Khomh. 2025. RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring. *arXiv preprint arXiv:2511.03153* (2025).
- [28] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. 2024. Next-generation refactoring: Combining llm insights and ide capabilities for extract method. In

- 2024 *IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 275–287.
- [29] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Andrey Sokolov, Timofey Bryksin, and Danny Dig. 2024. Em-assist: Safe automated extractmethod refactoring with llms. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 582–586.
 - [30] Xing Qian and Eman Abdullah Alomar. 2025. Can LLMs Identify and Refactor Code Clones? An Empirical Study. *Journal of Systems and Software* (2025), 112717.
 - [31] Chaiyong Ragkhitwetsagul, Jens Krinke, and David Clark. 2018. A comparison of code similarity analysers. *Empirical Software Engineering* 23, 4 (2018), 2464–2519.
 - [32] Jason E. Robbins and David F. Redmiles. 2000. Cognitive support, UML adherence, and XMI interchange in Argo/UML. *Information and Software Technology* 42, 2 (2000), 79–89. [https://doi.org/10.1016/S0950-5849\(99\)00083-X](https://doi.org/10.1016/S0950-5849(99)00083-X)
 - [33] Chanchal Kumar Roy and James R Cordy. 2007. A survey on software clone detection research. *Queen’s School of computing TR* 541, 115 (2007), 64–68.
 - [34] Chanchal K Roy, James R Cordy, and Rainer Koschke. 2009. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of computer programming* 74, 7 (2009), 470–495.
 - [35] Shahbaz Siddeeq, Muhammad Waseem, Zeeshan Rasheed, Md Mahade Hasan, Jussi Rasku, Mika Saari, Henri Terho, Kalle Mäkelä, Kai-Kristian Kemell, and Pekka Abrahamsson. 2025. LLM-Based Multi-agent System for Intelligent Refactoring of Haskell Code. In *International Conference on Product-Focused Software Process Improvement*. Springer, 408–418.
 - [36] Jeffrey Svajlenko, Judith F Islam, Iman Keivanloo, Chanchal K Roy, and Mohammad Mamun Mia. 2014. Towards a big data curated benchmark of inter-project code clones. In *2014 IEEE international conference on software maintenance and evolution*. IEEE, 476–480.
 - [37] Suresh Thummalapenta, Luigi Cerulo, Lerina Aversano, and Massimiliano Di Penta. 2010. An empirical study on the maintenance of source code clones. *Empirical Software Engineering* 15, 1 (2010), 1–34. <https://doi.org/10.1007/s10664-009-9108-x>
 - [38] Nikolaos Tsantalis, Davood Mazinanian, and Giri Panamoottil Krishnan. 2015. Assessing the refactorability of software clones. *IEEE Transactions on Software Engineering* 41, 11 (2015), 1055–1090.

- [39] Qinghua Xu, Guancheng Wang, Lionel Briand, and Kui Liu. 2025. Hallucination to Consensus: Multi-Agent LLMs for End-to-End Test Generation. *arXiv preprint arXiv:2506.02943* (2025).
- [40] Yisen Xu, Feng Lin, Jinqiu Yang, Nikolaos Tsantalis, et al. 2025. Mantra: Enhancing automated method-level refactoring with contextual rag and multi-agent llm collaboration. *arXiv preprint arXiv:2503.14340* (2025).
- [41] Zixian Zhang and Takfarinas Saber. 2024. Assessing the code clone detection capability of large language models. In *2024 4th International Conference on Code Quality (ICCCQ)*. IEEE, 75–83.